MateR-2.0.1 Usage

Javier Fernández-González

2026/06/11 (yyyy/mm/dd)

Contents

Introduction **1**

Self-Pollinated Crop **3**
Single Trait Optimization, Maximize Yield. 6
Single Trait Optimization, Maximize Yield, Add the Reference Population 9
Single Trait Optimization, Maximize Yield, Constrain the Number of Families 11
Single Trait Optimization, Maximize Yield, Constrain the Number of Crosses per Parent 13
Single Trait Optimization, Minimize Maturity 14
Multi-Trait Optimization 16
Using Haplotype Data 18
Disregard Linkage Disequilibrium 20
Comparison of Linkage Disequilibrium Approaches 21
Change optimization algorithm 21
Hyperparameter tuning 23
Double Haploids 26

Clonally Propagated, Autotetraploid Crop **27**
Computing Marker Effects from Breeding and Dominance Values 27
Multi-Trait Optimization 31
Using Haplotypes for Non-Homozygous Parents 33

Hybrid Crop **34**
Optimal Crosses Between Heterotic Pools for Hybrid Generation 37
Breeding Within a Heterotic Pool Given its Complementary Heterotic Pool 39

Post-Optimization and Optimal Recycling **42**
Post-Optimization, self-pollinated crop 44
Post-Optimization, reference population 46
Post-Optimization, hybrid crop 48

Advantages and Limitations **50**

References **51**

Introduction

This package allows to perform **optimal genomic mating** for optimal cross selection in a single trait or **multi-trait** framework. It is highly versatile, supporting breeding schemes used in **self-pollinated, clonal and hybrid crops**. It can consider the number of selfing cycles performed before selection, it can account for dominance effects and it can be used to breed to maximize the specific combining ability with a given set

of testers. It supports **diploid, allopolyploid and autotetraploid** species, as well as the use of double haploids. A detailed theoretical background is provided in Fernández-González et al. (2026) and extensive testing is performed in Metwally et al. (2025).

In genomic mating, there is a set of parental lines available and we want to answer the question of how to optimally combine them to obtain the best possible offspring. To that end, two criteria have to be balanced:

1. **Usefulness criterion**. This value combines the **family average** (the average genotypic value of all individuals generated by a cross) and the **within-family variance**. The more variance a family has, the more diverse it is and the higher the likelihood of producing exceptionally high-fitness individuals. Furthermore, a large number of offspring generated for a family allows for a better exploration of its diversity, although there are diminishing returns as this number increases. Selected families should ideally present a **high average and a high variance**. Moreover, the **number of offspring** generated per family should be **larger for high-variance families** than for low-variance ones, as in the former there are more potential gains from exploring its diversity. This can be done by repeating the crosses that generate high-variance families more times than the low-variance crosses.
2. **Across-family diversity**. Selecting diverse parents is key for keeping the genetic diversity of the population, which is essential for long-term gain. There is a trade-off between maximizing diversity and usefulness, as high diversity requires selecting a balanced mix of high performance and low performance parents. MateR package maximizes usefulness with the constraint that the diversity cannot drop below an user-defined threshold.

MateR **requires license keys**, but they will be provided for **free to public bodies**, such as, Universities, and non-profit organizations. You can get license keys by contacting javfer98@gmail.com or j.isidro@upm.es.

Please **make sure to have TrainSel installed before attempting to install MateR**. In the following sections, we will showcase the usage of MateR for different breeding schemes.

A note on the examples in this vignette. Throughout, the TrainSel-based optimizations use the demo settings (`size="demo"`) so that the vignette runs quickly. The demo version is limited and does not converge to an optimal solution, so all printed numerical results are illustrative only and will differ from a production run. For real analyses, use the recommended settings shown (commented out) next to each `control` definition. The Greedy algorithm does not use the `control` argument, so its results are unaffected by these settings.

```
#Load the library
library(MateR)
```

```
##
## ************************************************************
##                 This is MateR version 2.0.1
##
## TrainSel package is required
## Available in https://github.com/TheRocinante-lab/TrainSel
## To obtain license keys, please contact
## javfer98@gmail.com or j.isidro@upm.es.
## Free license keys will be provided to public bodies,
## such as, Universities, and non-profit organizations.
## ************************************************************
##
## Citation:
```

```
## Fernández-González J, Metwally SM, Isidro Y Sánchez J.
##  MateR: a novel genomic mating framework. Genetics. 2026 Jan 19:iyag013.
## doi: 10.1093/genetics/iyag013
set.seed(1234)
```

```
#To get license keys, please contact javfer98@gmail.com or j.isidro@upm.es
#They will be provided for free to public bodies, such as, Universities, and non-profit organizations
username <- NULL #type your username here.
password <- NULL #type your password here
username_TrainSel <- NULL #type your TrainSel username here.
password_TrainSel <- NULL #type your TrainSel password here
```

Self-Pollinated Crop

In a **self-pollinated crop**, there is typically a set of elite, **fully homozygous parental lines**. These parents are crossed among themselves to generate new variability. The resulting **F1 offspring are then repeatedly selfed** to increase their homozygosity. At some point during the selfing process, field trials are performed and the best genotypes are selected. MateR allows to find the best mating plan for **maximizing the usefulness of the offspring in the generation in which they are evaluated, i.e., after several cycles of selfing.**

First, we will load some example data:

```
#Load the example data
data(ExampleDataDiploid)

#1) There are 100 parental lines
length(Parents)
```

```
## [1] 100
```

```
Parents[1:5] #names of the parental lines
```

```
## [1] "g1_1" "g1_2" "g1_3" "g1_4" "g1_5"
```

```
#2) Genotypic information of the parents
#marker matrix counting the number of times the alternative allele is present in each locus
dim(Markers) #100 parental lines, 1000 Markers
```

```
## [1]  100 1000
```

```
Markers[1:5,1:5]
```

```
##      SNP1 SNP2 SNP3 SNP4 SNP5
## g1_1    0    2    0    2    0
## g1_2    0    0    0    0    0
## g1_3    0    0    0    0    0
## g1_4    0    0    0    0    0
## g1_5    0    0    0    0    0
```

```
sum(Markers == 1) #no heterozygous position. Fully inbred parental lines
```

```
## [1] 0
```

```
#3) Additive marker effects for two traits, yield (YLD) and maturity (MAT):
markereffects$YLD[1:5]
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
##  0.2288611 -1.0132430 -0.7770513 -0.4573172 -1.2113599
```

```
markereffects$MAT[1:5]
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
##  1.6079669  1.1523572 -0.2692306  0.2806039  1.4864272
```

```
#4) Coefficients for a multi-trait selection index
coefficients
```

```
##        YLD        MAT
##  0.9594875 -0.2817511
```

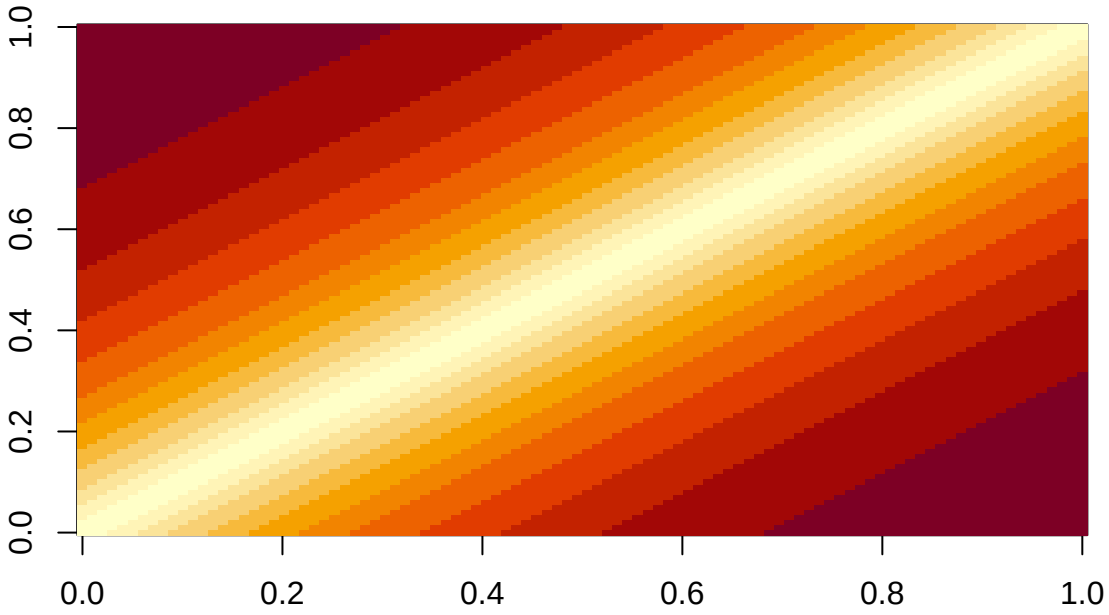
```
#5) List of matrices of frequencies of recombination per each chromosome
str(Chromosomes) #list indicating which markers belong to each chromosome
```

```
## List of 8
##  $ : chr [1:100] "SNP1" "SNP2" "SNP3" "SNP4" ...
##  $ : chr [1:150] "SNP101" "SNP102" "SNP103" "SNP104" ...
##  $ : chr [1:150] "SNP251" "SNP252" "SNP253" "SNP254" ...
##  $ : chr [1:100] "SNP401" "SNP402" "SNP403" "SNP404" ...
##  $ : chr [1:150] "SNP501" "SNP502" "SNP503" "SNP504" ...
##  $ : chr [1:150] "SNP651" "SNP652" "SNP653" "SNP654" ...
##  $ : chr [1:100] "SNP801" "SNP802" "SNP803" "SNP804" ...
##  $ : chr [1:100] "SNP901" "SNP902" "SNP903" "SNP904" ...
```

```
#assume all chromosomes have a length of 150 cM
ChromosomeCentimorgans <- rep(150, length(Chromosomes))
#Assume Haldane map function
c_list <- create_c_list(Chromosomes = Chromosomes,
                        ChromosomeCentimorgans = ChromosomeCentimorgans,
                        model = "Haldane")
c_list[[1]][1:5,1:5] #0 --> recombination never happens; 0.5 --> independent segregation
```

```
##            SNP1       SNP2       SNP3       SNP4       SNP5
## SNP1 0.00000000 0.01492425 0.02940303 0.04344964 0.05707698
## SNP2 0.01492425 0.00000000 0.01492425 0.02940303 0.04344964
## SNP3 0.02940303 0.01492425 0.00000000 0.01492425 0.02940303
## SNP4 0.04344964 0.02940303 0.01492425 0.00000000 0.01492425
## SNP5 0.05707698 0.04344964 0.02940303 0.01492425 0.00000000
```

```
image(c_list[[1]]) #Visualize one chromosome
```




```
#6) Haplotype data (optional)
H_parents[[1]][,1:5] #which allele is present in each chromosome
```

```
##            SNP1 SNP2 SNP3 SNP4 SNP5
## DH_gamete    0    1    0    1    0
## DH_gamete    0    1    0    1    0
```

Regarding the matrices of recombination frequencies `c_list`, it is important to note that, ideally, this information should be extracted from a linkage map for maximum accuracy. However, if the map is not available, a rough approximation can be made with the `create_c_list()` function. Nevertheless, `create_c_list()` relies on a few simplistic assumptions and it will incur in some error. For more details, please type `?create_c_list` in R.

Using this data, we can find the optimal mating plan for making crosses among the **100 parental lines**. As an example, we will assume the following parameters:

1. We have enough resources in our breeding program to **make 80 crosses**.
2. To ensure that enough diversity is retained, we want to ensure that no more than 5% of additive standard deviation is lost every selection cycle.
3. We will make the field trials to select the best **offspring in the F4** generation, i.e., after **3 cycles of selfing** from the original F1 offspring. As the final objective is obtaining fully inbred lines with no heterozygosity, we will focus on improving the additive breeding values of the individuals. **Dominance effects are not of interest** in this scheme as they are not heritable.
4. From **each cross** between two parental lines, we will obtain **20 different individuals in the F4**.
5. In the F4 generation, we will **select the best 5 individuals from each family** to continue the selfing process. The best individuals are selected through phenotypic selection. Thus, **selection accuracy**

will be the **square root of heritability**, specifically narrow sense heritability as only additive effects are of interest in this scheme.

6. We will consider that broad sense heritability is 0.3 for all traits and narrow sense heritability is 0.2. Heritability is very small due to the low replication of the field trials in which the best 5 individuals from each family would be selected (due to low seed availability).

We can create some parameters that reflect the information above:

```
#Demo TrainSel settings, for speed (see the note at the start of the vignette).
control = TrainSel::SetControlDefault(size="demo",
                                      verbose = F)

# #Recommended parameters (slower but converge to a much better solution):
# control = TrainSel::SetControlDefault(size="large",
#                                       complexity = "high_complexity",
#                                       verbose = F)

#limit diversity loss:
#limit of diversity loss = losing 5% of additive standard deviation
PropSD_limit <- 0.05
Number_of_crosses <- 80 #We are limited to less than 100 by the demo version!
Selfing_cycles <- 3 #3 cycles of selfing: we will perform selection in the F4
F4_individuals_per_cross <- 20
F4_selected_per_family <- 5
h2 <- 0.2
H2 <- 0.3
```

Single Trait Optimization, Maximize Yield.

We will start with a single-trait optimization. MateR by default maximizes a trait, making yield maximization easy:

```
#get the marker effects  for the desired trait
markereffectsYLD <- list(YLD = markereffects$YLD)

#Perform optimization to maximize yield with the desired parameters:
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       markereffects = markereffectsYLD, #marker effects for desired trait
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       c_list = c_list,
                       coefficients = NULL,
                       offspring_per_cross = F4_individuals_per_cross,
                       within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                       control = control,
                       n_selected_per_family=F4_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
```

```
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1  g1_1/g2_37    g1_1   g2_37                14     -18.862824
## 2  g1_3/g1_30    g1_3   g1_30                 7      -4.527680
## 3  g1_4/g1_28    g1_4   g1_28                 7      -3.970999
## 4 g1_11/g1_14   g1_11   g1_14                 7      15.236957
## 5 g1_14/g1_15   g1_14   g1_15                 3      20.118221
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            22.97875   4.115927
## 2                            18.26124  13.733559
## 3                            18.52634  14.555346
## 4                            17.72458  32.961536
## 5                            15.88885  36.007076
```

```
#diversity loss:
out$PropSD
```

```
## [1] 0.04988258
```

```
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 1.23113762073856e-07 i.e., one in 8122569"
```

```
#selection intensity from selection candidates to selected parents
out$i$selected_parents
```

```
## [1] 1.371905
```

One important thing to note about the `GenomicMatingMT()` function, is that it has two arguments for available parental lines, `Parents1` and `Parents2`. If these two arguments are different, the function will consider that the only crosses allowed are the ones involving one parent from `Parents1` and another parent from `Parents2`. In this case, we have set `Parents1=Parents2=Parents`. This means that **all crosses from a full diallel** of the lines in `Parents` **are allowed.**

The parameter `PropSD` is used to control how much diversity should be maintained. It is directly interpretable as the expected proportion of additive standard deviation lost in each selection cycle. As additive standard deviation is directly proportional to genetic gain, `PropSD` can be seen as the reduction in the rate of genetic gain per selection cycle due to dwindling diversity, allowing us to optimize its value. In case you are more familiar to the use of inbreeding rate to control for the diversity of the mating plan, MateR also supports it (`deltaF` parameter). By default, MateR computes inbreeding based on the off-diagonals of the genomic relationship matrix. Alternatively, if you set `deltaF_Endelman = TRUE`, the Endelman (2025) equations for inbreeding rate will be used. They are better suited when working with autopolyploids, but currently they are only supported if no inbreeding cycles (`n=0`) and no double haploids (`DHs=FALSE`) are performed.

```
inbreedingRate <- 0.05 #desired limit of 5% of inbreeding per generation:

#Perform optimization to maximize yield with the desired parameters:
out <- GenomicMatingMT(Parents1 = Parents,
```

```
                        Parents2 = Parents,
                        Markers = Markers,
                        parametrization = "Genotypic",
                        phi = 2, #Diploid or allopolyploid crop
                        markereffects = markereffectsYLD, #marker effects for desired trait
                        n = Selfing_cycles,
                        ############################################################
                        deltaF = inbreedingRate,#use inbreeding rate instead of PropSD
                        deltaF_Endelman = FALSE,#Compute inbreeding from off-diagonals of G
                        ############################################################
                        size = Number_of_crosses,
                        c_list = c_list,
                        coefficients = NULL,
                        offspring_per_cross = F4_individuals_per_cross,
                        within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                        control = control,
                        n_selected_per_family=F4_selected_per_family,
                        Username=username, #you can get one by contacting us
                        Password=password, #you can get one by contacting us
                        Username_TrainSel=username_TrainSel, #you can get one by contacting us
                        Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
## Warning in GenomicMatingMT(Parents1 = Parents, Parents2 = Parents, Markers =
## Markers, : As inbreeding rate tracks IBD similarity, we recommend working with
## a numerator relationship matrix instead of a genomic relationship matrix. You
## can provide it in the "G" argument. Computing inbreeding with the genomic
## relationships can work well if pedigree is not available, although it becomes
## difficult to interpret its value.
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1  g1_3/g1_33    g1_3   g1_33                5     -15.375326
## 2 g1_11/g2_33   g1_11   g2_33                7       2.668125
## 3 g1_14/g1_41   g1_14   g1_41                4      18.658240
## 4 g1_14/g2_49   g1_14   g2_49                5      14.356700
## 5  g1_15/g2_3   g1_15    g2_3                7     -12.609430
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            17.10852   1.733198
## 2                            19.30443  21.972556
## 3                            16.11931  34.777548
## 4                            19.97686  34.333557
## 5                            20.05996   7.450527
```

```
#the mating plan is below the desired threshold of 5% inbreeding rate:
out$deltaF
```

```
##            [,1]
## [1,] 0.04957452
```

```
#We can also see diversity loss that corresponds to this inbreeding rate
out$PropSD
```

```
## [1] 0.0435092
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 2.39159037018766e-09 i.e., one in 418131806"
```

```
#selection intensity from selection candidates to selected parents
out$i$selected_parents
```

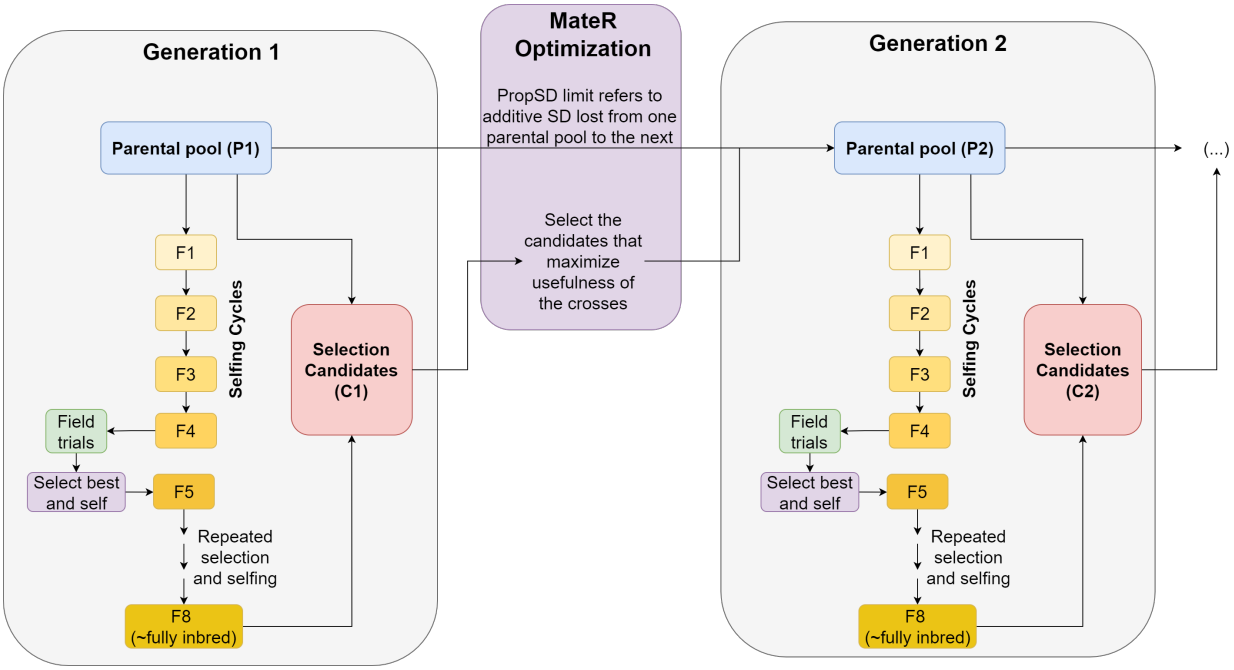
```
## [1] 1.260275
```

The main problem of working with inbreeding rate is that calculating inbreeding from the genomic relationship matrix incurs in some error. It should instead be computed from a numerator relationship matrix, which is not always available. If available, it can be provided to MateR through the `G` argument of `GenomicMatingMT()`.

In summary, we recommend using proportion of standard deviation lost to control for inbreeding in the population, as it can be calculated from any type of data and it is more easily interpretable.

Single Trait Optimization, Maximize Yield, Add the Reference Population

The `PropSD` parameter refers to the proportion of additive standard deviation lost due to selection when performing the mating plan. However, to properly interpret it, we need to know which two populations are being compared when `PropSD` is computed.



As outlined in Figure 1, optimization should select the best individuals among the candidates from the first generation (C1) to obtain a mating plan that maximizes usefulness and minimizes variance depletion, i.e., `PropSD`. To maximize interpretability, it is interesting for `PropSD` to measure the overall additive SD lost in an entire selection cycle. This can be done by comparing the same population in two adjacent generations. In the figure, it is done by comparing how much lower the additive SD is in P2 compared to P1. This can be done in MateR by setting a reference population that is equal to P1, and taking into account both the genotypes within P1 and how much they contributed to the offspring (i.e., unequal contributions).

If a reference population is not set, MateR computes PropSD using C1 as the reference population, i.e., it computes how much lower is the genetic variability in the selected P2 compared to the entire pool of candidates C1 with equal contributions. This is a useful workaround if the reference population is not known, as it still allows to control diversity loss, but it negatively impacts the interpretability of `PropSD`, as it is now

comparing two completely different populations from adjacent generations. As a result, without a reference population `PropSD` cannot be interpreted as the additive SD lost in an entire selection cycle, it would simply be the SD lost from the selection candidates to the selected parents.

As a result, we strongly recommend using the reference population whenever possible. Here, we will show an example of how it can be done, and we will keep using it for all subsequent analyses in this vignette.

```
#First, create a data.frame with the reference population
#Here we will just randomly sample it for the sake of the example, but in a real
#breeding program it has to be equal to the P1 population
Reference_pop <- data.frame(Genotypes = sample(rownames(Markers), 16),
                            Frequency = sample(1:4, 16, replace = T))
head(Reference_pop)
```

```
##   Genotypes Frequency
## 1     g1_42         1
## 2     g2_47         2
## 3     g1_13         3
## 4     g2_24         1
## 5     g1_28         3
## 6     g2_14         4
```

The reference population has to be a data.frame with two columns named "Genotypes" and "Frequency". Genotypes" contains the genotypic names, and the marker data of these genotypes must be provided to MateR. "Frequency" reflects the unequal contributions of the parents. Two options are supported, absolute or relative frequencies. In this case, we have absolute frequencies counting the number of crosses in which each parent participated.

```
#Perform optimization to maximize yield with the desired parameters:
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       ############################################################
                       Reference_population = Reference_pop, #set reference here
                       ############################################################
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       markereffects = markereffectsYLD, #marker effects for desired trait
                       n = Selfing_cycles,
                       ############################################################
                       PropSD = PropSD_limit, #go back to using PropSD
                       ############################################################
                       size = Number_of_crosses,
                       c_list = c_list,
                       coefficients = NULL,
                       offspring_per_cross = F4_individuals_per_cross,
                       within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                       control = control,
                       n_selected_per_family=F4_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1  g1_4/g1_14    g1_4   g1_14                 9      16.020950
## 2 g1_11/g1_34   g1_11   g1_34                10       9.162551
## 3 g1_14/g1_43   g1_14   g1_43                 8      21.481458
## 4 g1_15/g2_16   g1_15   g2_16                10      18.855186
## 5 g1_39/g2_49   g1_39   g2_49                13       3.679307
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            18.94212   34.96307
## 2                            19.09351   28.25606
## 3                            19.10914   40.59060
## 4                            21.28350   40.13868
## 5                            23.04296   26.72227
```

```
#deltaF is reported here but is NOT the constrained metric (we constrained PropSD).
#With a reference population, deltaF is measured against a different baseline, so a
#larger value is expected:
out$deltaF
```

```
##           [,1]
## [1,] 0.2049809
```

```
#PropSD is the metric we actually constrained, and it is below the desired limit:
out$PropSD
```

```
## [1] 0.03917556
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 1.45564614695282e-07 i.e., one in 6869801"
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 2.099341
```

Single Trait Optimization, Maximize Yield, Constrain the Number of Families

If a specific number of families is desired, it is possible to do it with the `n_families` parameter. For instance, if we want to have 150 individuals in the F4 after selection and we select 5 individuals per family, we need to have exactly 30 different families. We can do it as follows:

```
n_families <- 30 #specify desired number of families

#Perform optimization to maximize yield with the desired parameters:
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       markereffects = markereffectsYLD, #marker effects for desired trait
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
```

```
                        c_list = c_list,
                        coefficients = NULL,
                        offspring_per_cross = F4_individuals_per_cross,
                        within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                        control = control,
                        n_selected_per_family=F4_selected_per_family,
                        ############################################################
                        n_families = n_families, #specify desired number of families
                        ############################################################
                        Username=username, #you can get one by contacting us
                        Password=password, #you can get one by contacting us
                        Username_TrainSel=username_TrainSel, #you can get one by contacting us
                        Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##       Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1 g1_1/g1_11    g1_1   g1_11                2     -1.1359849
## 2 g1_2/g1_43    g1_2   g1_43                2      0.1345081
## 3  g1_3/g1_4    g1_3    g1_4                2      4.7150402
## 4 g1_3/g1_11    g1_3   g1_11                2      3.9310476
## 5 g1_3/g1_14    g1_3   g1_14                3     17.2216623
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            13.73372   12.59774
## 2                            13.94036   14.07487
## 3                            13.30235   18.01739
## 4                            12.54611   16.47716
## 5                            15.19589   32.41755
```

```
length(unique(out$OptimalMatingScheme$Family)) #There are exactly 30 families as desired
```

```
## [1] 30
```

```
#diversity loss :
out$PropSD
```

```
## [1] 0.04291991
```

```
#The value above is lower than the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 4.06680463826969e-06 i.e., one in 245893"
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 1.916291
```

Please note that adding more constraints may reduce the ability of the algorithm to converge to an optimal solution. We advise adding a constraint for the number of families only if it is really needed.

Single Trait Optimization, Maximize Yield, Constrain the Number of Crosses per Parent

Sometimes, due to practical reasons, a parent can only be crossed a limited number of times. This is specially evident in animal breeding, where females can often be crossed only once per cycle. The `Parents_limits` argument allows to introduce this constraint. Here, we will show an example in which the first 20 parents can participate in up to 4 crosses while the remaining 80 parents can only participate in 2 each:

```
Parents_limits <- data.frame(parents = Parents,
                             #minimum number of times each parent has to be used.
                             #We set it at zero (we don't force any parent to be chosen)
                             minimum = rep(0,length(Parents)),
                             maximum = c(rep(4,20), #maximum 4 crosses for first 20 parents
                                         rep(2,80))) #maximum 2 crosses for last 80 parents
head(Parents_limits)
```

```
##   parents minimum maximum
## 1    g1_1       0       4
## 2    g1_2       0       4
## 3    g1_3       0       4
## 4    g1_4       0       4
## 5    g1_5       0       4
## 6    g1_6       0       4
```

```
#Perform optimization to maximize yield with the desired parameters:
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       ############################################################
                       Parents_limits = Parents_limits,
                       opt_type = "Greedy", #generally better when Parents_limits exists
                       ############################################################
                       Markers = Markers,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       markereffects = markereffectsYLD, #marker effects for desired trait
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       c_list = c_list,
                       coefficients = NULL,
                       offspring_per_cross = F4_individuals_per_cross,
                       within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                       n_selected_per_family=F4_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1 g1_14/g2_16   g1_14   g2_16                 1       27.26454
## 2 g1_34/g1_42   g1_34   g1_42                 1       19.96521
```

```
## 3  g1_4/g2_16    g1_4   g2_16                1       14.75791
## 4 g1_14/g2_33   g1_14   g2_33                1       15.95874
## 5 g1_14/g1_15   g1_14   g1_15                1       20.11822
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            11.67480   38.93934
## 2                            10.68571   30.65092
## 3                            11.79415   26.55207
## 4                            12.38441   28.34315
## 5                            10.97779   31.09601
```

```
#diversity loss:
out$PropSD
```

```
## [1] -0.02029845
```

```
#The value above is negative, meaning the plan gains additive SD (a net diversity
#increase) rather than losing it: the parental limits force a more balanced,
#conservative plan, well below the diversity-loss limit
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 0.00105718050856207 i.e., one in 946"
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 0.9349732
```

```
#check that no parent is crossed more times than the limit
ParentsCount <- c()
scheme <- out$OptimalMatingScheme
for (parent in Parents_limits$parents) {
  tmp <- max(0, sum(scheme$Number_Of_Crosses[scheme$Parent1 == parent]))
  tmp <- tmp + max(0, sum(scheme$Number_Of_Crosses[scheme$Parent2 == parent]))

  ParentsCount <- c(ParentsCount, tmp)
}
names(ParentsCount) <- Parents_limits$parents

sum(ParentsCount > Parents_limits$maximum) #no parent is crossed more times than the limit
```

```
## [1] 0
```

Please note that adding more constraints may reduce the ability of the algorithm to converge to an optimal solution. Simultaneously adding the constraints of `n_families` and `Parents_limits` is allowed but not advised, as it is possible that no solution exists that meets both simultaneously while keeping across-family diversity above the desired threshold.

Single Trait Optimization, Minimize Maturity

Continuing with single-trait optimization, we will minimize maturity. As **MateR by default maximizes a trait**, we have to indicate that we want **to minimize** it. This can be easily done by **creating a coefficient (weight) of minus one for the trait**. This will result on the optimization maximizing the opposite of maturity, which is the same as minimizing it:

```r
#get the marker effects  for the desired trait
markereffectsMAT <- list(MAT = markereffects$MAT)
Coefficient <- c(MAT=-1) #give a weight of minus one to maturity to minimize it

#Perform optimization to minimize maturity with the desired parameters
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       markereffects = markereffectsMAT,#marker effects for desired trait
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       c_list = c_list,
                       ############################################################
                       coefficients = Coefficient, #negative coefficient: minimize trait
                       ############################################################
                       offspring_per_cross = F4_individuals_per_cross,
                       within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                       control = control,
                       n_selected_per_family=F4_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```r
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1 g1_46/g2_35   g1_46   g2_35               12       38.27691
## 2 g1_47/g2_24   g1_47   g2_24               14       16.99608
## 3 g2_14/g2_17   g2_14   g2_17                6       62.49946
## 4 g2_14/g2_27   g2_14   g2_27                7       61.47761
## 5 g2_14/g2_38   g2_14   g2_38                7       57.82993
##   Deviation_From_Average_Selected_Best Usefulness
## 1                             21.94925   60.22616
## 2                             23.69445   40.69053
## 3                             17.41367   79.91313
## 4                             19.52943   81.00704
## 5                             19.21863   77.04856
```

```r
#The expected breeding values are the opposite of the values in the table:
ExpectedMaturityValues <- -out$OptimalMatingScheme$Usefulness
ExpectedMaturityValues[1:5] #Very small values as desired, maturity has been minimized
```

```
## [1] -60.22616 -40.69053 -79.91313 -81.00704 -77.04856
```

```r
#diversity loss:
out$PropSD
```

```
## [1] 0.04924064
```

```
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 0.00346876624060777 i.e., one in 288"
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 1.068462
```

Multi-Trait Optimization

A more realistic scenario involves **selecting the offspring for maximum yield and minimum maturity simultaneously**. As a result, we want to be able to consider both traits when performing the genomic mating. To that end, we need to work within the framework of **index selection**: we have some coefficients that correspond to the weight of each trait. We will maximize the index scores, which are simply the weighted sum of the traits.

Getting the best values for the coefficients can be difficult. One possible way of getting them is the `gain()` function from StageWise package. Details on how to use it are available in Vignette3 of StageWise (https://github.com/jendelman/StageWise). Here, we will just use the coefficients available in the example data.

```
#Multi-trait coefficients:
#Positive value for yield: we want to maximize it
#Negative value for maturity: we want to minimize it
#Weight for yield has a larger magnitude than the one for maturity. This means
#that yield has more economical importance than maturity.
coefficients
```

```
##        YLD        MAT
##  0.9594875 -0.2817511
```

```
markereffects$YLD[1:5] #additive marker effects for yield
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
##  0.2288611 -1.0132430 -0.7770513 -0.4573172 -1.2113599
```

```
markereffects$MAT[1:5] #additive marker effects for maturity
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
##  1.6079669  1.1523572 -0.2692306  0.2806039  1.4864272
```

```
#both traits are correlated, making multi-trait analysis more important
cor(markereffects$YLD, markereffects$MAT)
```

```
## [1] 0.2663142
```

```
#Perform optimization to maximize the index scores (multi-trait):
start <- Sys.time()
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       Reference_population = Reference_pop, #set reference here
```

```
                        parametrization = "Genotypic",
                        phi = 2, #Diploid or allopolyploid crop
                        markereffects = markereffects, #marker effects for both traits
                        n = Selfing_cycles,
                        PropSD = PropSD_limit,
                        size = Number_of_crosses,
                        c_list = c_list,
                        ############################################################
                        coefficients = coefficients, #coefficients for index scores
                        ############################################################
                        offspring_per_cross = F4_individuals_per_cross,
                        within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                        control = control,
                        n_selected_per_family=F4_selected_per_family,
                        Username=username, #you can get one by contacting us
                        Password=password, #you can get one by contacting us
                        Username_TrainSel=username_TrainSel, #you can get one by contacting us
                        Password_TrainSel=password_TrainSel #you can get one by contacting us
)
end <- Sys.time()
timeLD <- end-start #save this for later
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1  g1_4/g1_14    g1_4   g1_14                8       2.421977
## 2 g1_11/g1_43   g1_11   g1_43                4       2.701577
## 3 g1_14/g1_46   g1_14   g1_46                8      15.812060
## 4 g1_15/g1_35   g1_15   g1_35                9      -3.094020
## 5 g1_15/g1_46   g1_15   g1_46                7      13.081445
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            17.32344   19.74541
## 2                            14.47240   17.17398
## 3                            17.96855   33.78061
## 4                            17.66225   14.56823
## 5                            18.89079   31.97224
```

```
#diversity loss:
out$PropSD
```

```
## [1] 0.04792927
```

```
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 6.67164606205084e-06 i.e., one in 149888"
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 1.846749
```

```
#save this for later:
Family_values_LD <- out$FamilyValues
```

It is important to note that the **usefulness values in the output** correspond to the **multi-trait index scores**.

Using Haplotype Data

Computing **within-family variance** requires taking into account the **linkage disequilibrium (LD)** of the parental lines. This is controlled in the `GenomicMatingMT()` function through the `LD` argument, which defaults to `LD="Approx"`. This results in the assumption that the **LD pattern of any parental line is well represented by the LD computed on the overall parental population**. `LD="Approx"` combines very fast computational **speed** with good **performance**, but it can incur in some error. For instance, if the parental lines present **strong population structure**, the phasing patterns in the haplotypes may be different in each subpopulation. Therefore, the LD of a random parent would be well represented by the overall LD patterns of its subpopulation, but not necessarily by the LD patterns in the entire population. Alternatively, MateR allows to accurately calculate the **LD patterns of any specific parental line from its haplotype matrix** (setting `LD="Full"`). This is completely **robust to the population structure** in the parental lines and makes very consistent and **highly accurate predictions**. However, it presents three main **disadvantages**: 1) **haplotype data has to be available** (which is trivial for fully homozygous parental lines but requires phasing information otherwise), 2) it is more **computationally intensive** and 3) it is less robust to high error in the estimation of marker effects (which is often the case in empirical datasets). Next, we will show how to use the haplotypes:

Compute haplotype data internally

This option is only available **if the parental lines are fully homozygous**. If this is the case, we simply need to set `LD="Full"` and phasing information will be computed by MateR:

```
sum(Markers == 1) #no heterozygous positions in parental lines --> fully homozygous
```

```
## [1] 0
```

```
LD="Full" #Use haplotypes to fully account for linkage disequilibrium
```

```
#Perform optimization to maximize yield with the desired parameters:
start <- Sys.time()
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       c_list = c_list,
                       ############################################################
                       LD=LD, #Fully account for linkage disequilibrium
                       ############################################################
                       markereffects = markereffectsYLD, #marker effects for desired trait
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       coefficients = NULL,
                       offspring_per_cross = F4_individuals_per_cross,
                       within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                       control = control,
```

```
                      n_selected_per_family=F4_selected_per_family,
                      Username=username, #you can get one by contacting us
                      Password=password, #you can get one by contacting us
                      Username_TrainSel=username_TrainSel, #you can get one by contacting us
                      Password_TrainSel=password_TrainSel #you can get one by contacting us
)
end <- Sys.time()
timeHaplotypes <- end-start #save this for later
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1  g1_3/g1_34    g1_3   g1_34                 9      11.147256
## 2  g1_7/g1_43    g1_7   g1_43                 8       1.780034
## 3 g1_14/g1_39   g1_14   g1_39                 6      17.850179
## 4 g1_14/g1_41   g1_14   g1_41                 6      18.658240
## 5 g1_14/g2_33   g1_14   g2_33                12      15.958740
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            17.84447   28.99173
## 2                            17.54981   19.32984
## 3                            15.03918   32.88936
## 4                            13.91006   32.56829
## 5                            26.39227   42.35101
```

```
#diversity loss:
out$PropSD
```

```
## [1] 0.04985053
```

```
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 1.7012281271489e-06 i.e., one in 587811"
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 1.967671
```

```
#save this for later:
Family_values_Haplotypes <- out$FamilyValues
```

This approach allows to make extremely accurate predictions but it is slower. It is possible to **parallelize** it by indicating the desired number of cores to use in the argument `ncores` of the `GenomicMatingMT()` function. However, it is important to note that **there is some overhead involved** in the parallelization, which can reduce its efficiency in some scenarios.

Explicitly provide haplotype data

MateR also allows to **manually provide phasing information** for parental lines through the `H_parents` argument of `GenomicMatingMT()` function. This is **only needed if the parental lines are not fully**

homozygous. An example of this will be provided for the autotetraploid, clonally propagated crop (see the **"Using Haplotypes for Non-Homozygous Parents"** section).

Disregard Linkage Disequilibrium

Above, we have shown how to compute within-family variance with haplotypes or without haplotypes using the LD patterns of the entire parental population. Both approaches perform well and it is recommended to use one of them. Nevertheless, if a simpler approach is desired, MateR supports a final alternative. **Within-family variance can be calculated assuming no LD** (independent loci). This is rather **simplistic** and generally has **substantially more error** that the other approaches. However, it has the advantages of **not needing** to know any information about the **recombination frequencies** in the genome and it is the **fastest methodology**.

```
#Example for Haplotype data unavailable:
LD="Ind" #do not use linkage disequilibrium of the parents as an input for computing
#within-family variance.

#Perform optimization to maximize yield with the desired parameters:
start <- Sys.time()
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       ############################################################
                       c_list = NULL, #No need for recombination frequencies
                       LD=LD, #LD="Ind" --> disregard LD
                       ############################################################
                       markereffects = markereffectsYLD, #marker effects for desired trait
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       coefficients = NULL,
                       offspring_per_cross = F4_individuals_per_cross,
                       within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                       control = control,
                       n_selected_per_family=F4_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
end <- Sys.time()
timeNoLD <- end-start #save this for later
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1  g1_3/g2_48    g1_3   g2_48                10     -14.803705
## 2 g1_11/g1_43   g1_11   g1_43                 8       8.190844
## 3 g1_14/g1_42   g1_14   g1_42                 8      26.039619
## 4 g1_14/g1_46   g1_14   g1_46                 9      18.357239
## 5 g1_34/g1_42   g1_34   g1_42                10      19.965213
```

```
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            23.14600   8.342291
## 2                            18.72719  26.918033
## 3                            19.83174  45.871361
## 4                            19.75581  38.113052
## 5                            19.68005  39.645259
```

```
#diversity loss:
out$PropSD
```

```
## [1] 0.04846893
```

```
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 4.00553884838395e-07 i.e., one in 2496543"
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 2.046646
```

```
#save this for later:
Family_values_no_LD <- out$FamilyValues
```

Comparison of Linkage Disequilibrium Approaches

Let's now compare the three LD approaches available in MateR in terms of computational time and performance:

```
#computational time:
timeHaplotypes #time required when using haplotypes to compute LD
```

```
## Time difference of 38.30549 secs
```

```
timeLD #time required when assuming that LD of a parent is equal to LD in the population
```

```
## Time difference of 21.05779 secs
```

```
timeNoLD #time required when disregarding LD
```

```
## Time difference of 19.24852 secs
```

As expected, using **haplotypes is slower**, but its time requirements are in the same order of magnitude as its alternatives. For a performance comparison, please refer to the MateR paper.

Change optimization algorithm

MateR supports two different optimization algorithms to find the best mating plan. The first one, is **TrainSel**, which combines a genetic algorithm with simulated annealing. It is very flexible and powerful, containing tools that allow it to avoid getting stuck in local optima. It works very well for small or intermediate datasets (around 100 parental lines or less). However, for larger datasets it becomes very slow and its ability to find the global optimum can get significantly degraded due to the massive number of possible combinations that it has to go through. Alternatively, MateR also contains a built-in **Greedy** algorithm that starts by selecting the best family and then iteratively adds the next best one that meets existing constraints. This is much

faster and works much better in high dimensionality scenarios (hundreds or thousands of parental lines), but it is prone to getting stuck in a local optimum, making it often worse than TrainSel for relatively small datasets. Its speed also makes it very well suited for rapidly making initial tests, while moving to TrainSel only for the final solution.

MateR by default automatically chooses between TrainSel and the Greedy algorithm, but it is possible to manually choose one of them through the `opt_type` argument. Here we leave an example:

```
#LD="Approx" (the default): represent each parent's LD by the LD of the overall
#parental population. This does not require haplotype data.
LD="Approx"

out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       Reference_population = Reference_pop, #set reference here
                       ##########################################################
                       opt_type = "Greedy", #manually choose the greedy algorithm
                       #opt_type = "TrainSel", #manually choose TrainSel
                       #opt_type = "Auto", #default, MateR chooses the best algorithm
                       ##########################################################
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       c_list = c_list,
                       LD=LD, #LD="Approx"
                       markereffects = markereffectsYLD, #marker effects for desired trait
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       coefficients = NULL,
                       offspring_per_cross = F4_individuals_per_cross,
                       within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                       n_selected_per_family=F4_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1 g1_14/g2_16   g1_14   g2_16                 1       27.26454
## 2 g1_34/g1_42   g1_34   g1_42                 3       19.96521
## 3  g1_4/g2_16    g1_4   g2_16                11       14.75791
## 4 g1_14/g2_33   g1_14   g2_33                13       15.95874
## 5 g1_43/g2_16   g1_43   g2_16                 3       20.21842
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            11.67480   38.93934
## 2                            15.46611   35.43133
## 3                            22.02372   36.78164
## 4                            23.72690   39.68564
## 5                            15.94224   36.16067
```

```r
#diversity loss:
out$PropSD
```

```
## [1] 0.0497894
```

```r
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```r
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 9.98873369573161e-08 i.e., one in 10011279"
```

```r
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 2.17825
```

As you may have noticed, the greedy algorithm is much faster than TrainSel. In this example it also performed better (we know that through the `out$Optimization_quality` value), but the reason for that is that we are currently using the demo parameters for TrainSel, which substantially drag its performance down. With the full version of TrainSel, we would expect it to outperform the greedy algorithm for a dataset of this size, although at the cost of a greatly increased computational time.

Hyperparameter tuning

Until now, we have assumed that the desired **parameters** such as the **size of the mating plan** and the **limit for proportion of standard deviation lost** are known. However, if these values are not known with certainty, it may be useful to perform some testing on how they would impact the overall mating plan.

The optimization pipeline can be divided into **two computationally intensive steps**:

1. Compute the average value of each family and **within-family variance**. This step is independent from the optimization hyperparameters and it is often the computational bottleneck when using haplotype data.
2. Perform **optimization** to maximize usefulness.

When doing repeated tests, it is advised to **perform the first step only once** and then **repeatedly run the second step** with different hyperparameters to perform the tuning. The first step can be manually calculated using the `calculateFamilyValues()` function or it can be **extracted from the output of a previous optimization**. We will use the latter option, as it is generally easier. Furthermore, we will also use the **Greedy** optimization algorithm to **maximize the speed** of the second step. We will show how to test different values of the `PropSD` limit:

```r
#extract family mean and sd from the previous optimization
FamilyValues <- Family_values_Haplotypes
FamilyValues$mean[1:5]
```

```
## $`g1_1/g1_2`
## [1] -9.19232
##
## $`g1_1/g1_3`
## [1] 0.8487202
##
## $`g1_1/g1_4`
## [1] -0.3519923
```

```
##
## $`g1_1/g1_5`
## [1] -33.01788
##
## $`g1_1/g1_6`
## [1] -9.099269
```

```
FamilyValues$sd[1:5]
```

```
## $`g1_1/g1_2`
## [1] 20.90436
##
## $`g1_1/g1_3`
## [1] 16.37305
##
## $`g1_1/g1_4`
## [1] 18.38269
##
## $`g1_1/g1_5`
## [1] 15.85391
##
## $`g1_1/g1_6`
## [1] 19.12954
```

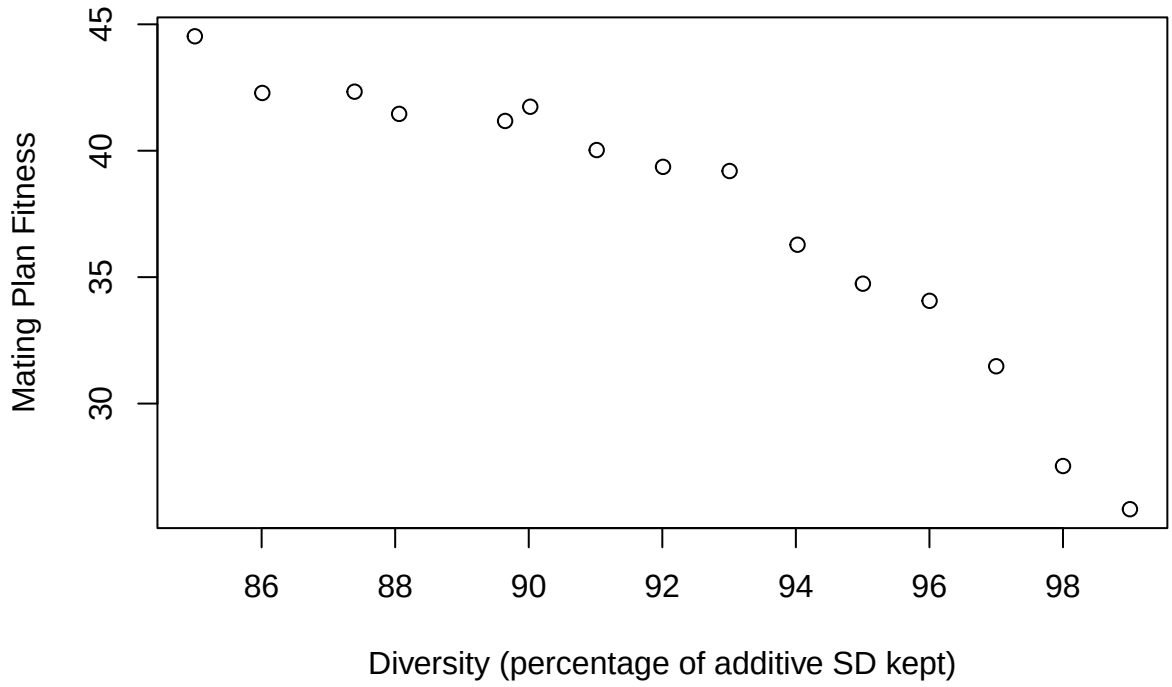
```
#Test different values for the limit of diversity loss
PropSD_values <- seq(0.01, 0.15, by = 0.01)
PropSD_values
```

```
## [1] 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.10 0.11 0.12 0.13 0.14
0.15
```

```
#Perform tests with different values of the parameters and using the precomputed
#FamilyValues to increase speed:
mating_plan_PropSD <- c()
mating_plan_fitness <- c()
for (PropSD_value in PropSD_values) {
  suppressWarnings(out <- GenomicMatingMT(Parents1 = Parents,
                        Parents2 = Parents,
                        FamilyValues = FamilyValues, #Include here precomputed values!
                        Markers = Markers,
                        Reference_population = Reference_pop, #set reference here
                        ############################################################
                        opt_type = "Greedy", #manually choose the greedy algorithm
                        ############################################################
                        parametrization = "Genotypic",
                        phi = 2, #Diploid or allopolyploid crop
                        markereffects = NULL, #not needed here: FamilyValues are precomputed
                        n = Selfing_cycles,
                        ############################################################
                        PropSD = PropSD_value, #test several values for PropSD
                        ############################################################
                        size = Number_of_crosses,
                        offspring_per_cross = F4_individuals_per_cross,
                        within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                        n_selected_per_family=F4_selected_per_family,
                        Username=username, #you can get one by contacting us
```

```
                      Password=password, #you can get one by contacting us
                      Username_TrainSel=username_TrainSel, #you can get one by contacting us
                      Password_TrainSel=password_TrainSel #you can get one by contacting us
  ))
  mating_plan_PropSD <- c(mating_plan_PropSD, out$PropSD)
  mating_plan_fitness <- c(mating_plan_fitness, out$Fitness)
}

#Explore trade-off between diversity and gain.
#We used the Greedy algorithm here, which does not rely on the "control" argument,
#so the demo TrainSel settings have no effect on these results. Any small
#irregularities in the curve come from the greedy heuristic settling on slightly
#different local optima across PropSD values, not from the demo settings.
Percentage_SD_saved <- (1 - mating_plan_PropSD)*100
plot(x = Percentage_SD_saved,
     y = mating_plan_fitness,
     xlab = "Diversity (percentage of additive SD kept)",
     ylab = "Mating Plan Fitness")
```



As expected, mating plans with lower diversity allow to obtain more short-term gain (higher fitness) and vice versa. From this plot, it seems that values for the proportion of additive standard deviation lost of around 0.07 (7%) present a good balance between the two factors. Which PropSD value is the optimal one depends of the priorities of the breeding program: larger values are preferred for aggressive programs aiming for large short-term gain, while lower values are preferred for long-term sustainability. The **key advantage of PropSD** over other diversity metrics is that it can be directly **interpreted within the breeder's equation**. For a given PropSD value, additive standard deviation after n selection cycles (σ_{an}) is expected

to be reduced to $E(\sigma_{an}) = (1 - PropSD)^n \sigma_{a0}$, where σ_{a0} is the initial additive standard deviation. In the breeder's equation ($R = ir\sigma_{an}$), genetic gain (R) is directly proportional to σ_{an}. This allows to make **future projections** about how much genetic gain will be affected by the depletion of genetic variability in the future for a given PropSD value, which is extremely useful for finding its optimal value for a given time-horizon.

It is important to note that here we performed optimization 15 times using the within-family variance computed with haplotype data. If within-family variance had to be computed every single time, it would have been substantially slower.

Double Haploids

MateR also supports using double haploids (DHs). Here, we will show how we would perform optimization for a breeding program similar to the previous ones but with DHs performed on the F1 generation instead of performing selfing cycles:

```
DHs=TRUE #DHs are performed
Selfing_cycles=0 #0 selfing cycles before the DHs (i.e., the DHs are performed on the F1)
```

```
#Perform optimization to maximize yield with the desired parameters:
out <- GenomicMatingMT(Parents1 = Parents,
                       Parents2 = Parents,
                       Markers = Markers,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       c_list = c_list,
                       LD="Approx",
                       ############################################################
                       DHs = DHs, #We are using DHs
                       n = Selfing_cycles, #no selfing cycles
                       ############################################################
                       markereffects = markereffectsYLD, #marker effects for desired trait
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       coefficients = NULL,
                       offspring_per_cross = F4_individuals_per_cross,
                       within_family_accuracy = sqrt(h2), #narrow sense (only additive)
                       control = control,
                       n_selected_per_family=F4_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1 g1_12/g2_16   g1_12   g2_16               10      -7.013616
## 2 g1_14/g1_15   g1_14   g1_15                3      20.118221
## 3 g1_14/g1_23   g1_14   g1_23                5      10.310666
## 4 g1_14/g1_32   g1_14   g1_32                9      11.148987
## 5 g1_14/g1_34   g1_14   g1_34                3      22.453166
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            21.73510   14.72148
```

```
## 2                              16.89815   37.01637
## 3                              17.99532   28.30598
## 4                              21.43300   32.58199
## 5                              16.55585   39.00902
```

```
#diversity loss:
out$PropSD
```

```
## [1] 0.04917572
```

```
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 5.10176433543563e-07 i.e., one in 1960106"
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 2.019878
```

Clonally Propagated, Autotetraploid Crop

In a **clonally propagated crop**, there is no need to get fully homozygous lines because the plant material can be **asexually propagated** without altering its genotype. As a result, breeding in this kind of crops involves crossing the available elite lines to generate new variability, selecting the best and propagating them clonally. This also makes **dominance effects relevant**, as they can improve the phenotype and **they are not lost when propagating clonally.**

Next, we will also consider that in this example the **crop is autotetraploid** to showcase how the package handles it. Finally, the **example data available for this crop does not contain directly the marker effects**. Instead, it has additive and non-additive genotypic values for each line. **We will also show how the marker effects can be extracted** from them for their use in genomic mating.

Computing Marker Effects from Breeding and Dominance Values

First, we will load the example data. **The genotypic values available** are similar to what could be obtained from the following **GBLUP model:**

$$\boldsymbol{y} = \mathbf{1}\mu + \boldsymbol{F}b + Z_a\boldsymbol{a} + Z_d\boldsymbol{d} + \boldsymbol{\epsilon}$$

Where $\boldsymbol{y}$ is a vector of phenotypes, μ is a fixed intercept, $\boldsymbol{F}$ is a vector of inbreeding coefficients (computed as the diagonal of the VanRaden genomic relationship matrix minus one, divided by the ploidy level minus one, i.e., $(\text{diag}(G) - 1)/(\phi - 1)$), b is a regression coefficient indicating the inbreeding effect (directional dominance), $\boldsymbol{a} \sim N(\mathbf{0}, G\sigma^2_a)$ is a vector of additive breeding values, $\boldsymbol{d} \sim N(\mathbf{0}, D\sigma^2_d)$ is a vector of dominance values and $\boldsymbol{\epsilon} \sim N(\mathbf{0}, I\sigma^2_\epsilon)$ is a vector of residuals. $\mathbf{1}$, Z_a and Z_d are design matrices for their corresponding effects. G is an additive relationship matrix, D is a dominance relationship matrix and I is the identity matrix with the needed dimensions. σ^2_a and σ^2_d are variance components estimated by the model.

```
rm(list = ls()) #remove everything from the R environment
```

```
data(ExampleDataAutotetraploid)
```

```r
#1) There are 100 parental lines
length(Parents4)
```

```
## [1] 100
```

```r
Parents4[1:5]
```

```
## [1] "l_1" "l_2" "l_3" "l_4" "l_5"
```

```r
#2) Genotypic information of the parents
#marker matrix counting the number of times the alternative allele is present in each locus
Markers4[1:5,1:5]
```

```
##     SNP1 SNP2 SNP3 SNP4 SNP5
## l_1    1    2    2    2    1
## l_2    2    0    0    0    0
## l_3    0    1    2    2    2
## l_4    2    1    1    0    0
## l_5    1    2    1    1    0
```

```r
phi <- max(Markers4) #autotetraploids: ploidy degree = 4

#3) Additive breeding values (a) for two traits, yield (YLD) and maturity (MAT):
AVs4$YLD[1:5] #we will need to convert these into marker effects
```

```
##      l_1      l_2      l_3      l_4      l_5
## 22.43147 86.02203 17.25468 10.52742 40.14012
```

```r
AVs4$MAT[1:5] #we will need to convert these into marker effects
```

```
##       l_1       l_2       l_3       l_4       l_5
##  4.948174 67.428844 -55.446234 12.011615 40.066247
```

```r
#4) Dominance values (d) for two traits, yield (YLD) and maturity (MAT):
DVs4$YLD[1:5] #we will need to convert these into marker effects
```

```
##       l_1       l_2       l_3       l_4       l_5
## -30.272322  1.998564 -41.930402  2.757924 61.560330
```

```r
DVs4$MAT[1:5] #we will need to convert these into marker effects
```

```
##       l_1       l_2       l_3       l_4       l_5
##  5.997176 20.190428 -52.055431 -2.850171 -1.553687
```

```r
#5) Directional dominance (inbreeding effect):
G4[1:5,1:5] #VanRaden relationship matrix
```

```
##              l_1          l_2          l_3          l_4         l_5
## l_1  9.094108e-01 -0.076627398 -0.078758041  6.289246e-05  0.03926158
## l_2 -7.662740e-02  0.967862276  0.004375518 -5.670718e-02  0.26614932
## l_3 -7.875804e-02  0.004375518  0.898141496 -7.680709e-02 -0.01450505
## l_4  6.289246e-05 -0.056707176 -0.076807091  9.004775e-01 -0.01782937
## l_5  3.926158e-02  0.266149323 -0.014505054 -1.782937e-02  0.94421985
```

```r
#Extract inbreeding coefficients (F) from the VanRaden relationship matrix
Fcoeff <- (diag(G4)-1)/(phi-1)

#inbreeding effect (b) for yield.
#As it is negative, the crop presents inbreeding depression and heterosis
```

```
b4$YLD
```

```
## [1] -2
```

```
#Directional dominance values for yield:
(Fcoeff*b4$YLD)[1:5]
```

```
##        l_1        l_2        l_3        l_4        l_5
## 0.06039280 0.02142515 0.06790567 0.06634833 0.03718677
```

```
#inbreeding effect (b) for maturity.
#As it is negative, the crop presents inbreeding depression and heterosis
b4$MAT
```

```
## [1] -1
```

```
#Directional dominance values for maturity:
(Fcoeff*b4$MAT)[1:5]
```

```
##        l_1        l_2        l_3        l_4        l_5
## 0.03019640 0.01071257 0.03395283 0.03317417 0.01859338
```

```
#6) Coefficients for a multi-trait selection index
coefficients
```

```
##        YLD        MAT
##  0.9594875 -0.2817511
```

```
#7) List of Matrices of frequencies of recombination per chromosome
c_list4[[1]][1:5,1:5]
```

```
##            SNP1       SNP2       SNP3       SNP4       SNP5
## SNP1 0.00000000 0.01492425 0.02940303 0.04344964 0.05707698
## SNP2 0.01492425 0.00000000 0.01492425 0.02940303 0.04344964
## SNP3 0.02940303 0.01492425 0.00000000 0.01492425 0.02940303
## SNP4 0.04344964 0.02940303 0.01492425 0.00000000 0.01492425
## SNP5 0.05707698 0.04344964 0.02940303 0.01492425 0.00000000
```

First, we will convert the additive and dominance values for the parental lines into marker effects. This can be easily done with the function `meff_from_GVs()`:

```
##############################################################################
#additive effects
##############################################################################
meffsYLD <- meff_from_GVs(Markers4,
                          parametrization = "Genotypic",
                        phi=4, #tetraploid. If diploid, just replace 4 with 2
                        GVs = AVs4$YLD, #breeding values
                        type="additive",
                        DirectionalDominance=NULL)
```

```
#Marker effects computed by the function allow to reconstruct the original values
max(abs(Markers4%*%meffsYLD - AVs4$YLD))
```

```
## [1] 3.268497e-13
```

```
meffsMAT <- meff_from_GVs(Markers4,
                          parametrization = "Genotypic",
                        phi=4, #autotetraploid. If diploid, just replace 4 with 2
                        GVs = AVs4$MAT, #breeding values
```

```
                        type="additive",
                        DirectionalDominance=NULL)
```

```
#Marker effects computed by the function allow to reconstruct the original values
max(abs(Markers4%*%meffsMAT - AVs4$MAT))
```

```
## [1] 4.831691e-13
```

```
#Store the marker effects in the format needed for MateR:
markereffects <- list(YLD = meffsYLD,
                      MAT = meffsMAT)


##############################################################################
#dominance effects
##############################################################################
#dominance incidence matrix
#for diploids this is 1 at heterozygous loci and 0 otherwise; for the autotetraploid
#here it takes integer values reflecting the heterozygosity configuration
#We compute this only for validation, the function meff_from_GVs computes it internally
Markers_dom <- Compute_Q(M = Markers4, phi = 4, parametrization = "Genotypic")
Markers4[1:5,1:5] #additive marker matrix
```

```
##     SNP1 SNP2 SNP3 SNP4 SNP5
## l_1    1    2    2    2    1
## l_2    2    0    0    0    0
## l_3    0    1    2    2    2
## l_4    2    1    1    0    0
## l_5    1    2    1    1    0
```

```
Markers_dom[1:5,1:5] #dominance marker matrix
```

```
##     SNP1 SNP2 SNP3 SNP4 SNP5
## l_1    3    4    4    4    3
## l_2    4    0    0    0    0
## l_3    0    3    4    4    4
## l_4    4    3    3    0    0
## l_5    3    4    3    3    0
```

```
FbYLD <- Fcoeff*b4$YLD #directional dominance effect
meffsYLD <- meff_from_GVs(Markers4, #additive marker matrix even if type = "dominance"!
                          parametrization = "Genotypic",
                        phi=4, #autotetraploid. If diploid, just replace 4 with 2
                        GVs = DVs4$YLD, #dominance values
                        type="dominance",
                        DirectionalDominance=FbYLD)
```

```
#The markers computed combine the original dominance values plus the effect
#of the directional dominance
max(abs(Markers_dom%*%meffsYLD -
      (DVs4$YLD + FbYLD)))
```

```
## [1] 3.446132e-13
```

```
FbMAT <- Fcoeff*b4$MAT #directional dominance effect
meffsMAT <- meff_from_GVs(Markers4, #additive marker matrix even if type = "dominance"!
                          parametrization = "Genotypic",
                        phi=4, #tetraploid. If diploid, just replace 4 with 2
```

```
                        GVs = DVs4$MAT, #dominance values
                        type="dominance",
                        DirectionalDominance=FbMAT)
```

```
#The markers computed combine the original dominance values plus the effect
#of the directional dominance
max(abs(Markers_dom%*%meffsMAT -
      (DVs4$MAT + FbMAT)))
```

```
## [1] 4.121148e-13
```

```
#Store the marker effects in the format needed for MateR:
markereffects_d <- list(YLD = meffsYLD,
                        MAT = meffsMAT)
```

It is important to note that the **dominance marker effects computed here have absorbed both the main dominance values** (***d*** in the model) **and the directional dominance** (***Fb*** in the model).

Multi-Trait Optimization

Now, we can peform the **genomic mating itself**. For the sake of simplicity, we will use **similar parameters as in the example from the self-pollinated** crop with a few exceptions: **dominance effects** will be considered and there will **not be any selfing** of the offspring. We also need to **specify that the crop is autotetraploid.**

```
#First, create a data.frame with the reference population
#Here we will just randomly sample it for the sake of the example, but in a real
#breeding program it has to be equal to the P1 population
Reference_pop <- data.frame(Genotypes = sample(rownames(Markers4), 16),
                            Frequency = sample(1:4, 16, replace = T))
head(Reference_pop)
```

```
##   Genotypes Frequency
## 1      l_27         1
## 2      l_21         4
## 3      l_79         1
## 4      l_31         2
## 5     l_100         2
## 6      l_36         1
```

```
#Demo TrainSel settings, for speed (see the note at the start of the vignette).
control = TrainSel::SetControlDefault(size="demo",
                                      verbose = F)

# #Recommended parameters (slower but converge to a much better solution):
# control = TrainSel::SetControlDefault(size="large",
#                                       complexity = "high_complexity",
#                                       verbose = F)


#limit diversity loss:
#limit of diversity loss = losing 5% of additive standard deviation
PropSD_limit <- 0.05
Number_of_crosses <- 80 #We are limited to less than 100 by the demo version!
Selfing_cycles <- 0 #No selfing in clonally propagated crops!
F1_individuals_per_cross <- 20
F1_selected_per_family <- 5
```

```r
phi = 4 #4 indicates autotetraploid crop
H2 = 0.3
h2 = 0.2
```

```r
#Perform optimization to maximize the index scores (multi-trait)
#Take LD into account but do not use haplotypes for computational efficiency
out <- GenomicMatingMT(Parents1 = Parents4,
                       Parents2 = Parents4,
                       Markers = Markers4,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       phi = phi, #Indicate that the crop is tetraploid!
                       LD="Approx",
                       markereffects = markereffects, #additive marker effects
                       markereffects_d = markereffects_d, #dominance marker effects
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       c_list = c_list4,
                       coefficients = coefficients, #coefficients for index scores
                       offspring_per_cross = F1_individuals_per_cross,
                       within_family_accuracy = sqrt(H2), #broad sense (a+d)
                       control = control,
                       n_selected_per_family=F1_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```r
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##      Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1  l_5/l_59     l_5    l_59                 9       89.76992
## 2 l_21/l_34    l_21    l_34                 9       94.14852
## 3 l_21/l_48    l_21    l_48                11      114.18354
## 4 l_48/l_76    l_48    l_76                10      116.08676
## 5 l_48/l_91    l_48    l_91                10      127.23138
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            20.58291   110.3528
## 2                            20.51341   114.6619
## 3                            20.23270   134.4162
## 4                            19.31586   135.4026
## 5                            19.87610   147.1075
```

```r
#diversity loss:
out$PropSD
```

```
## [1] 0.04795473
```

```r
#The value above is below the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```r
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 0.00104889925936325 i.e., one in 953. WARNING: this value
becomes meaningless in the presence of heterosis."
```

```r
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 1.262417
```

Using Haplotypes for Non-Homozygous Parents

As the parental lines are not fully homozygous in a clonally propagated crop, if we want to set `LD="Full"`, we need to manually provide the phasing information:

```r
H_parents4[[1]][,1:5] #phasing information
```

```
##      SNP1 SNP2 SNP3 SNP4 SNP5
## [1,]    0    1    1    1    1
## [2,]    1    0    0    0    0
## [3,]    0    1    1    1    0
## [4,]    0    0    0    0    0
```

```r
LD="Full"
```

```r
#Perform optimization to maximize the index scores (multi-trait)
#Take LD into account but do not use haplotypes for computational efficiency
out <- GenomicMatingMT(Parents1 = Parents4,
                       Parents2 = Parents4,
                       Markers = Markers4,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       phi = phi, #Indicate that the crop is tetraploid!
                       ############################################################
                       LD=LD, #LD="Full" --> Use haplotype information
                       H_parents = H_parents4, #We need to manually provide haplotypes!
                       ############################################################
                       markereffects = markereffects, #additive marker effects
                       markereffects_d = markereffects_d, #dominance marker effects
                       n = Selfing_cycles,
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       c_list = c_list4,
                       coefficients = coefficients, #coefficients for index scores
                       offspring_per_cross = F1_individuals_per_cross,
                       within_family_accuracy = sqrt(H2), #broad sense (a+d)
                       control = control,
                       n_selected_per_family=F1_selected_per_family,
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##      Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1 l_21/l_85    l_21    l_85                 9      110.19726
## 2 l_43/l_48    l_43    l_48                10       97.88161
## 3 l_44/l_48    l_44    l_48                10      110.70104
## 4 l_44/l_83    l_44    l_83                 9      102.53915
## 5 l_48/l_91    l_48    l_91                10      127.23138
##   Deviation_From_Average_Selected_Best Usefulness
## 1                             17.43144   127.6287
## 2                             19.14651   117.0281
## 3                             17.10247   127.8035
## 4                             17.77759   120.3167
## 5                             17.73863   144.9700
```

```
#diversity loss:
out$PropSD
```

```
## [1] 0.04861328
```

```
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 0.00102969758340166 i.e., one in 971. WARNING: this value
becomes meaningless in the presence of heterosis."
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 1.34891
```

Hybrid Crop

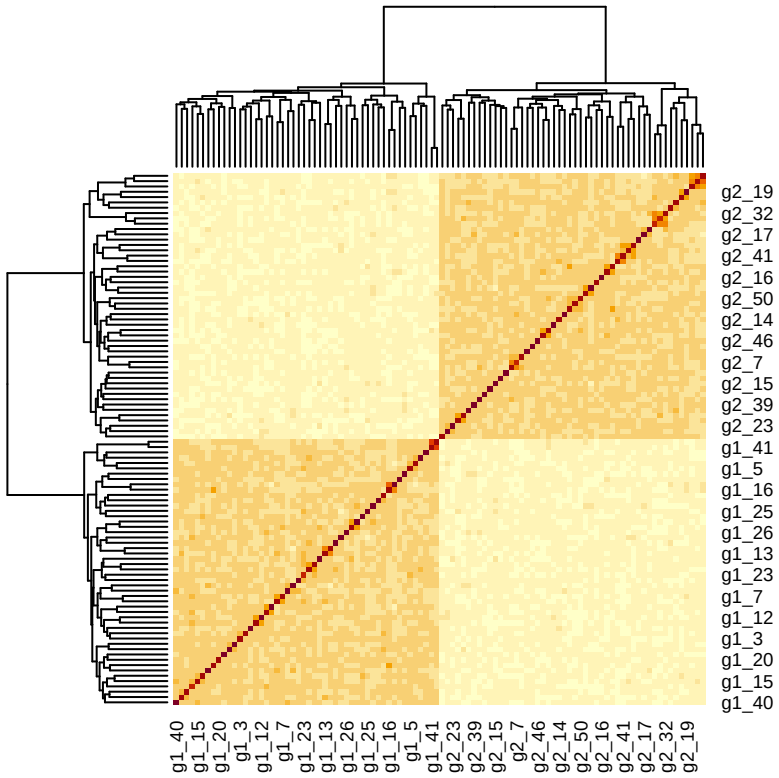
In **hybrids**, fully inbred **parental lines from complementary heterotic groups** are crossed to generate **highly heterozygous hybrids that display heterosis**, enhancing their performance. As such, considering **dominance effects is critical** for hybrids. For this example, we will go back to the diploid dataset. Actually, this dataset has two distinct subpopulations, which allows us to divide it into two different heterotic groups. This will allow us to showcase two ways to leverage genomic mating for hybrid breeding.

```
rm(list = ls()) #remove everything from the R environment
```

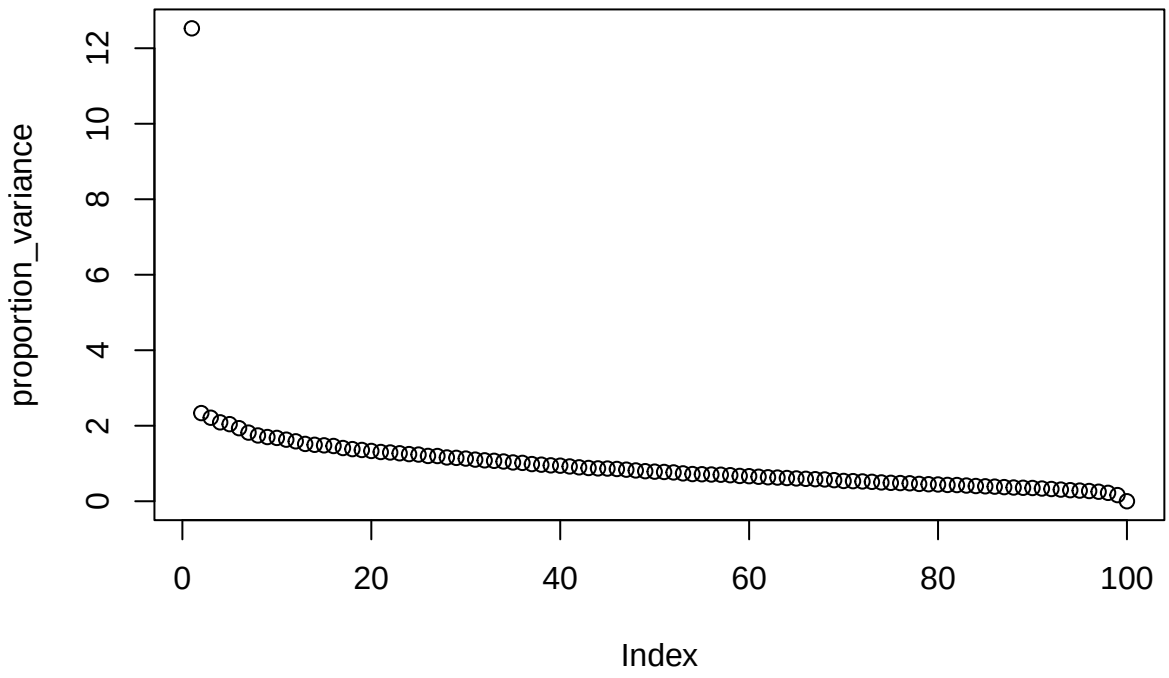
```
#Load the example data
data(ExampleDataDiploid)


#Explore population structure:
#Check population structure in the overall population
#We can clearly see two subpopulations:
#The first 50 lines have stronger relationships among themselves than with the
#remaining 50 and vice-versa
```

```
heatmap(G)
```




```
#get proportion of variance explained by the first principal components
#The first principal component explains far more variance than any other (over 12% of the total)!
#That confirms that the population structure is strong
eigenvalues <-eigen(G)$values
proportion_variance <- (eigenvalues/sum(eigenvalues))*100
plot(proportion_variance)
```



From the description of the dataset (available by typing `?ExampleDataDiploid` in R), we know that the individuals in the first group have the names "g1_1" through "g1_50" and the second group has the names "g2_1" through "g2_50". We will divide the data into two heterotic groups along these lines.

```
#1) There are 100 parental lines divided into two groups.
#They can be seen as two heterotic groups
Parents_g1 <- paste0("g1_",1:50) #First heterotic group
Parents_g2 <- paste0("g2_",1:50) #Second heterotic group
Parents_g1[1:5]
```

```
## [1] "g1_1" "g1_2" "g1_3" "g1_4" "g1_5"
```

```
Parents_g2[1:5]
```

```
## [1] "g2_1" "g2_2" "g2_3" "g2_4" "g2_5"
```

```
#2) Genotypic information of the parents
#marker matrix counting the number of times the alternative allele is present in each locus
Markers_g1 <- Markers[Parents_g1,]
Markers_g2 <- Markers[Parents_g2,]
rm(Markers)

#3) Additive marker effects for two traits, yield (YLD) and maturity (MAT):
markereffects$YLD[1:5]
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
##  0.2288611 -1.0132430 -0.7770513 -0.4573172 -1.2113599
```

```
markereffects$MAT[1:5]
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
##  1.6079669  1.1523572 -0.2692306  0.2806039  1.4864272
```

```
#4) Dominance marker effects for two traits, yield (YLD) and maturity (MAT):
markereffects_d$YLD[1:5]
```

```
##      SNP1      SNP2      SNP3      SNP4      SNP5
## 0.9850114 0.4712876 1.3134464 1.1436437 1.3044140
```

```
markereffects_d$MAT[1:5]
```

```
##        SNP1        SNP2        SNP3        SNP4        SNP5
##  1.59984818  0.02891102  1.26850629  1.07151226 -0.08323753
```

```
#5) Coefficients for a multi-trait selection index
coefficients
```

```
##        YLD        MAT
##  0.9594875 -0.2817511
```

```
#6) Haplotypes and recombination matrix
#Store separately the haplotypes of the two heterotic groups
H_Parents_g1 <- H_parents[Parents_g1]
H_Parents_g2 <- H_parents[Parents_g2]
rm(H_parents)
c_list[[1]][1:5,1:5]
```

```
##            SNP1       SNP2       SNP3       SNP4       SNP5
## SNP1 0.00000000 0.01492425 0.02940303 0.04344964 0.05707698
## SNP2 0.01492425 0.00000000 0.01492425 0.02940303 0.04344964
## SNP3 0.02940303 0.01492425 0.00000000 0.01492425 0.02940303
## SNP4 0.04344964 0.02940303 0.01492425 0.00000000 0.01492425
## SNP5 0.05707698 0.04344964 0.02940303 0.01492425 0.00000000
```

Optimal Crosses Between Heterotic Pools for Hybrid Generation

In this example, we want to **find the best possible crosses to perform between parents from the two groups** to obtain **hybrids with maximum performance**. As in the hybrids heterozygosity and low inbreeding is desired, **no selfing** is performed after crossing the parents. Furthermore, **dominance effects** have to be included to account specific combining ability and heterosis. Finally, as hybrids are a **F1 from two fully homozygous parents**, they are completely **homogeneous** and their **within-family variance is zero**. This has two implications:

1. Performing several crosses for the same family does not result in any advantage for maximizing usefulness. Thus, we have to **disallow making several crosses per family.**
2. Considering **LD** is only required for **computing within-family variance** and including it **would increase computational burden**. As we know that within-family **variance will be zero** for all possible F1 crosses, **we can completely disregard LD** to minimize the computational time without loss of accuracy.

```
#Demo TrainSel settings, for speed (see the note at the start of the vignette).
control = TrainSel::SetControlDefault(size="demo",
                                      verbose = F)

# #Recommended parameters (slower but converge to a much better solution):
# control = TrainSel::SetControlDefault(size="large",
```

```
#                                       complexity = "high_complexity",
#                                       verbose = F)


#When possible, keep the same parameters as for previous examples
#limit diversity loss:
#limit of diversity loss = losing 5% of additive standard deviation
PropSD_limit <- 0.05
Number_of_crosses <- 80 #We are limited to less than 100 by the demo version!
Selfing_cycles <- 0 #No selfing for hybrids!
F1_individuals_per_cross <- 20
F1_selected_per_family <- 5
Several_Crosses_per_family <- FALSE #Do not cross a pair of parents more than once
h2 = 0.2
H2 = 0.3
```

```
LD="Ind" #within family variance will always be zero --> use the fastest method

#Perform optimization to maximize the index scores (multi-trait)
#Disregard LD for computational efficiency. As within-family variance will always
#be zero, this has no impact on performance
out <- GenomicMatingMT(Parents1 = Parents_g1, #heterotic pool 1
                       Parents2 = Parents_g2, #heterotic pool 2
                       Markers = rbind(Markers_g1, Markers_g2),
                       ############################################################
                       Reference_population = NULL, #not needed here
                       ############################################################
                       parametrization = "Genotypic",
                       phi = 2, #Diploid or allopolyploid crop
                       markereffects = markereffects, #additive marker effects
                       markereffects_d = markereffects_d, #dominance marker effects
                       n = Selfing_cycles,
                       ############################################################
                       LD=LD, #LD="Ind" --> Disregard LD completely
                       ############################################################
                       PropSD = PropSD_limit,
                       size = Number_of_crosses,
                       coefficients = coefficients, #coefficients for index scores
                       offspring_per_cross = F1_individuals_per_cross,
                       within_family_accuracy = sqrt(H2), #broad sense (a+d)
                       control = control,
                       n_selected_per_family=F1_selected_per_family,
                       ############################################################
                       replication = Several_Crosses_per_family, #Only 1 cross per family
                       ############################################################
                       Username=username, #you can get one by contacting us
                       Password=password, #you can get one by contacting us
                       Username_TrainSel=username_TrainSel, #you can get one by contacting us
                       Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##       Family Parent1 Parent2 Number_Of_Crosses Family_average
```

```
## 1 g1_1/g2_16    g1_1   g2_16                1      -3.560300
## 2 g1_2/g2_16    g1_2   g2_16                1      -8.525958
## 3 g1_2/g2_33    g1_2   g2_33                1     -14.694946
## 4  g1_4/g2_1    g1_4    g2_1                1     -15.737992
## 5 g1_7/g2_14    g1_7   g2_14                1      -1.949426
##   Deviation_From_Average_Selected_Best Usefulness
## 1                                   0  -3.560300
## 2                                   0  -8.525958
## 3                                   0 -14.694946
## 4                                   0 -15.737992
## 5                                   0  -1.949426
```

```
#diversity loss:
out$PropSD
```

```
## [1] 0.0223466
```

```
#The value above is well below the desired limit; the constraints of this scheme
#(one cross per family across two heterotic pools) force a more conservative plan
PropSD_limit
```

```
## [1] 0.05
```

```
#how well optimization worked
out$Optimization_quality
```

```
## [1] "The likelihood of a solution better than the current one and with
similar diversity is 2.82077894375732e-09 i.e., one in 354512005. WARNING: this
value becomes meaningless in the presence of heterosis."
```

```
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 0.998821
```

The column "Deviation_From_Average_Selected_Best" always has a value of zero for all families because they are a homogeneous F1. All crosses are performed only once because we disallowed repeating them.

Regarding the parents of each family, in the function `GenomicMatingMT()`, we specified two different pools of parents. Therefore, the two parents of a family will always come from different pools. As seen in the output, the first parent will always belong to the first heterotic group (lines "g1_1" through "g1_50") and the second parent will belong to the second heterotic pool (lines "g2_1" through "g2_50").

Breeding Within a Heterotic Pool Given its Complementary Heterotic Pool

When breeding parental lines **within a heterotic pool**, crosses are made between the parental lines and the **offspring are selfed** to make them fully inbred, similarly to what happens in self-pollinated crops. However, the aim in this case is not maximizing the usefulness of these individuals themselves, but rather **maximizing the usefulness of the hybrids obtained when they are crossed with another heterotic pool**. To showcase how to breed within a heterotic pool through genomic mating, we will use the following scheme as an example:

1. **Crosses** are made between the parents of the first **heterotic pool**.
2. The resulting **F1 offspring are selfed to increase homozygosity**.
3. For instance, after **3 cycles of selfing (F4)**, the families will be **crossed with testers** from the second heterotic pool to **generate hybrids**. The **families will be evaluated according to the usefulness of the hybrids they produce.**

In summary, we will breed within the first heterotic pool, using the second one as testers:

```
head(Parents_g1) #heterotic group 1
```

```
## [1] "g1_1" "g1_2" "g1_3" "g1_4" "g1_5" "g1_6"
```

```
head(Parents_g2) #heterotic group 2 will be used as testers
```

```
## [1] "g2_1" "g2_2" "g2_3" "g2_4" "g2_5" "g2_6"
```

```
#Please note that generally it is advised to not use many testers for
#computational reasons.

#Create a data.frame with the reference population
#As we are looking at crosses within heterotic group 1, the reference population
#also contains parents exclusively from that group
#Here we will just randomly sample it for the sake of the example, but in a real
#breeding program it has to be equal to the P1 population
Reference_pop <- data.frame(Genotypes = sample(Parents_g1, 16),
                            Frequency = sample(1:4, 16, replace = T))
head(Reference_pop)
```

```
##   Genotypes Frequency
## 1      g1_7         1
## 2     g1_35         3
## 3     g1_31         4
## 4      g1_3         1
## 5      g1_5         3
## 6      g1_9         1
```

The `GenomicMatingMT()` function allows to **include genotypic information about the testers** that will be used. As a result, a mating plan that maximizes the usefulness of the hybrids can be generated:

```
#Keep all parameters as before with a few exceptions:
Selfing_cycles <- 3 #3 cycles of selfing before crossing with testers

#Please, note that the TestersApproach argument allows to control how the information
#of the testers is used when computing the usefulness of each cross:

# TestersApproach <- "max" #the usefulness of each family will be the one it obtains
# #when crossed with the tester most suitable for it

TestersApproach <- "average" #the usefulness of each family will be the average
#usefulness obtained when crossed with all available testers. This is the default value
```

```
#Perform optimization to maximize the index scores (multi-trait)
#Breed within the heterotic group comprised of the lines in "Parents_g1"
#Take LD into account but do not use haplotypes for computational efficiency (default)
out <- GenomicMatingMT(Parents1 = Parents_g1, #heterotic pool 1
                       Parents2 = Parents_g1, #heterotic pool 1.
                       Markers = Markers_g1,
                       Reference_population = Reference_pop, #set reference here
                       parametrization = "Genotypic",
                       ############################################################
                       Markers_T = Markers_g2, #Marker matrix of the testers!
                       TestersApproach = TestersApproach, #either "average" or "max"
                       LD = "Approx", #faster than LD="Full"
```

```
                        ############################################################
                        phi = 2, #Diploid or allopolyploid crop
                        markereffects = markereffects, #additive marker effects
                        markereffects_d = markereffects_d, #dominance marker effects
                        n = Selfing_cycles,
                        PropSD = PropSD_limit,
                        size = Number_of_crosses,
                        c_list = c_list,
                        coefficients = coefficients, #coefficients for index scores
                        offspring_per_cross = F1_individuals_per_cross,
                        within_family_accuracy = sqrt(H2), #broad sense (a+d)
                        control = control,
                        n_selected_per_family=F1_selected_per_family,
                        Username=username, #you can get one by contacting us
                        Password=password, #you can get one by contacting us
                        Username_TrainSel=username_TrainSel, #you can get one by contacting us
                        Password_TrainSel=password_TrainSel #you can get one by contacting us
)
```

```
#output
out$OptimalMatingScheme[1:5,] #summary of optimal mating scheme
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Recommended_Tester
## 1  g1_7/g1_14    g1_7   g1_14                 7              g2_49
## 2  g1_7/g1_15    g1_7   g1_15                 7              g2_16
## 3 g1_14/g1_15   g1_14   g1_15                 8              g2_49
## 4 g1_14/g1_41   g1_14   g1_41                 9               g2_4
## 5 g1_14/g1_42   g1_14   g1_42                 8              g2_16
##   Family_average Deviation_From_Average_Selected_Best Usefulness
## 1      -22.43379                            16.42221  -6.011580
## 2      -29.93447                            15.97531 -13.959167
## 3      -26.90828                            16.87558 -10.032695
## 4      -30.19428                            16.47067 -13.723617
## 5      -22.20078                            17.33731  -4.863469
```

```
#diversity loss:
out$PropSD
```

```
## [1] 0.04975462
```

```
#The value above is very close to the desired limit:
PropSD_limit
```

```
## [1] 0.05
```

```
#likelihood of a solution with better gain and similar diversity than the current
#mating plan:
#how well optimization worked.
#out$Optimization_quality #NOTE: the stronger the heterosis, the less meaningful this becomes!
#selection intensity from the reference population to selected parents
out$i$selected_parents
```

```
## [1] 1.459883
```

Some things to note in this scenario:

1. The **output** for the optimal mating scheme **includes the "Recommended_Tester"** column, indicating the best tester for each family.

2. The **usefulness values** in the output correspond to the **usefulness of the hybrids** resulting from crossing the F4 families with the testers.

3. In the arguments of the function `GenomicMatingMT()`, we have set `Parents1=Parents_g1` and `Parents2=Parents_g1`. This means that we are exploring which are the **best possible crosses among the full diallel of the parents** in `Parents_g1`, i.e., we are breeding within **the first heterotic group**.

4. We need to compute the within-family variance for all combinations between the testers and the full diallel of the first heterotic group. This can be **very computationally intensive**, specially **when using haplotype data**. Reducing the number of lines in the group of testers can greatly reduce the computational burden. Therefore, for large datasets, we **advise selecting only the most interesting individuals** from the second heterotic group **for their use as testers**.

5. If **DHs** are used, they can be specified following the same steps specified for the self-pollinated crops. Firstly, set `DHs = TRUE`. Secondly, set `n` to be the number of selfing cycles before the DHs, e.g., `n=0` if DHs are performed on the F1, `n=1` if DHs are performed on the F2, etc.

Post-Optimization and Optimal Recycling

Actually implementing the optimal mating plans can be challenging in some breeding schemes. A good **mating scheme** involves generating a **relatively large number of families** and selecting **several offspring from them** in the **initial screening** for further characterization and selection, which is reflected by the GenomicMatingMT() function. However, **often only the best few lines among the individuals selected in the initial screening are reused as parental lines for the next generation**. Simply selecting these genotypes by **truncation selection** on the available offspring would be **problematic**. Genomic mating carefully selects families with maximum genetic diversity. Simply selecting the best performing offspring in a later stage would neglect this and **waste most of the diversity of the mating plan**. As a result, we have developed a tool we called **"post-optimization"** that allows to **select a subset of the available offspring** that i) **maximizes genetic value** and ii) has the **desired genetic diversity**. This is similar to the concept of **Optimal recycling** in Metwally et al. (2025).

The dataset "ExampleMatingPlan" contains an example for this. It is the output of the following process:

1. Using the datset "ExampleDataDiploid", genomic mating was performed with the GenomicMatingMT() function. We used the following parameters: Parental lines were the lines g1_1 through g1_50 (first subpopulation). Haplotypes were used to compute their family variance. 80 crosses with 20 offspring per cross were desired. Only additive marker effects were considered, using the marker effects and multi-trait coefficients from "ExampleDataDiploid" dataset. It was considered that 5 offspring per family would be selected after 3 cycles of selfing. The best 5 offspring would be selected by phenotypic selection with a heritability of 0.25 (accuracy of 0.5, the square root of heritability). The selected offspring would be subsequently genotyped, i.e., their markers are available. The cutoff for the proportion of additive standard deviation lost (`PropSD`) was set to 0.05.

2. The optimal mating plan contained 14 unique families, with varying numbers of crosses per family.

3. All offspring from the optimal mating plan were created. We simulated both their markers and their phenotypic values following the desired heritability.

4. The initial screening was performed. The 5 individuals from each family with the highest phenotype were selected.

```
rm(list = ls()) #remove everything from the R environment
```

```
data(ExampleMatingPlan)
```

```
#1) Optimal mating plan:
GenomicMatingOutput$OptimalMatingScheme[1:5,]
```

```
##        Family Parent1 Parent2 Number_Of_Crosses Family_average
## 1  g1_4/g1_43    g1_4   g1_43                8       2.946254
## 2 g1_14/g1_15   g1_14   g1_15                5      13.685203
## 3 g1_14/g1_34   g1_14   g1_34                3      15.782364
## 4 g1_14/g1_42   g1_14   g1_42                4      17.395103
## 5 g1_14/g1_46   g1_14   g1_46                2      15.812060
##   Deviation_From_Average_Selected_Best Usefulness
## 1                            18.37688   21.32314
## 2                            18.28990   31.97510
## 3                            13.10109   28.88345
## 4                            17.35998   34.75509
## 5                            14.27559   30.08765
```

```
#14 different families generated
length(unique(GenomicMatingOutput$OptimalMatingScheme$Family))
```

```
## [1] 14
```

```
#80 total crosses
sum(GenomicMatingOutput$OptimalMatingScheme$Number_Of_Crosses)
```

```
## [1] 80
```

```
#2) Parental lines of the mating plan:
rownames(Markers_P)[1:5]
```

```
## [1] "g1_1" "g1_2" "g1_3" "g1_4" "g1_5"
```

```
#3) Lines that could be used as testers in a hybrid breeding program
rownames(Markers_potential_testers)[1:5]
```

```
## [1] "g2_1" "g2_2" "g2_3" "g2_4" "g2_5"
```

```
#4) Additive marker effects
markereffects$YLD[1:5]
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
##  0.2288611 -1.0132430 -0.7770513 -0.4573172 -1.2113599
```

```
markereffects$MAT[1:5]
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
##  1.6079669  1.1523572 -0.2692306  0.2806039  1.4864272
```

```
#5) Dominance marker effects
markereffects_d$YLD[1:5]
```

```
##      SNP1      SNP2      SNP3      SNP4      SNP5
## 0.9850114 0.4712876 1.3134464 1.1436437 1.3044140
```

```
markereffects_d$MAT[1:5]
```

```
##        SNP1        SNP2        SNP3        SNP4        SNP5
##  1.59984818  0.02891102  1.26850629  1.07151226 -0.08323753
```

```
#6) Multi-trait selection index coefficients
coefficients
```

```
##        YLD        MAT
##  0.9594875 -0.2817511
```

```
#7) Markers of the selected offspring (best 5 from each family); first 6 rows shown:
Markers_Offspring[1:6,1:10]
```

```
##                SNP1 SNP2 SNP3 SNP4 SNP5 SNP6 SNP7 SNP8 SNP9 SNP10
## g1_4/g1_43.119    2    0    0    0    0    0    0    0    0     0
## g1_4/g1_43.127    2    0    0    0    0    0    0    0    0     0
## g1_4/g1_43.2      0    0    0    0    0    0    0    2    0     0
## g1_4/g1_43.144    0    0    0    0    0    0    0    2    0     0
## g1_4/g1_43.93     1    0    0    0    0    0    0    1    0     0
## g1_14/g1_15.20    2    0    0    2    2    2    2    0    0     0
```

```
#70 individuals in total (14 families times 5 individuals selected per family)
dim(Markers_Offspring)
```

```
## [1]   70 1000
```

```
#8) Link between each individual name, the family to which it belongs and its parents
SelectedKeyTable[1:6,]
```

```
##   Parent1 Parent2      Family      Offspring
## 3    g1_4   g1_43  g1_4/g1_43 g1_4/g1_43.119
## 4    g1_4   g1_43  g1_4/g1_43 g1_4/g1_43.127
## 1    g1_4   g1_43  g1_4/g1_43   g1_4/g1_43.2
## 5    g1_4   g1_43  g1_4/g1_43 g1_4/g1_43.144
## 2    g1_4   g1_43  g1_4/g1_43  g1_4/g1_43.93
## 7   g1_14   g1_15 g1_14/g1_15 g1_14/g1_15.20
```

```
#Create multi-trait marker effects for index scores:
markereffectsAll <- rep(0, ncol(Markers))
markereffectsAll_d <- rep(0, ncol(Markers))
for (trait in names(coefficients)) {
  markereffectsAll <- markereffectsAll + markereffects[[trait]]*coefficients[trait]

  if (!is.null(markereffects_d)) {
    markereffectsAll_d <- markereffectsAll_d + markereffects_d[[trait]]*coefficients[trait]
  }

}
markereffectsAll[1:5] #additive
```

```
##       SNP1       SNP2       SNP3       SNP4       SNP5
## -0.2334571 -1.2968719 -0.6697150 -0.5178506 -1.5810872
```

```
markereffectsAll_d[1:5] #dominance
```

```
##      SNP1      SNP2      SNP3      SNP4      SNP5
## 0.4943472 0.4440488 0.9028323 0.7954121 1.2750212
```

Using this dataset, we will **showcase** how **post-optimization** can be performed in a **self-pollinated** crop and in a **hybrid** crop. It could also be used in a similar manner for **clonally propagated** crops.

Post-Optimization, self-pollinated crop

In this scenario, we will assume that, **from the available 70 offspring, we want to select the best 15** while maintaining enough diversity. As we are in a **self-pollinated** crop, we are only interested in maximizing

additive breeding values and we will ignore dominance.

```
phi = 2 #diploid crop
size = 15 #we want to select 15 individuals
#limit diversity loss:
PropSD_limit <- 0.05

#Demo TrainSel settings, for speed (see the note at the start of the vignette).
control = TrainSel::SetControlDefault(size="demo",
                                      verbose = F)

# #Recommended parameters (slower but converge to a much better solution):
# control = TrainSel::SetControlDefault(size="large",
#                                       complexity = "high_complexity",
#                                       verbose = F)
```

```
set.seed(12)
out <- PostOpt(Markers_Cand = Markers_Offspring,
               parametrization = "Genotypic",
               phi = phi,
               size = size,
               markereffects = markereffects,
               markereffects_d = NULL, #self-pollinated --> no dominance
               coefficients = coefficients,
               PropSD = PropSD_limit,
               control = control,
               Username=username, #you can get one by contacting us
               Password=password, #you can get one by contacting us
               Username_TrainSel=username_TrainSel, #you can get one by contacting us
               Password_TrainSel=password_TrainSel) #you can get one by contacting us)
```

```
PropSD_limit #desired limit to diversity loss
```

```
## [1] 0.05
```

```
out$PropSD #below the desired cutoff
```

```
## [1] 0.04319319
```

```
#Selected individuals
out$OptimalSelection #the Fitness column is simply the breeding value
```

```
##         Genotypes  Fitness
## 1  g1_14/g1_15.20 60.79553
## 2  g1_14/g1_15.13 61.74161
## 3  g1_14/g1_34.54 49.25167
## 4  g1_14/g1_42.43 51.25888
## 5  g1_14/g1_42.41 51.05556
## 6  g1_14/g1_42.57 59.96783
## 7   g1_14/g1_46.5 52.78535
## 8  g1_14/g1_46.28 56.40305
## 9  g1_15/g1_43.36 45.66105
## 10 g1_34/g1_43.62 46.83346
## 11 g1_34/g1_43.39 48.48654
## 12 g1_39/g1_42.65 50.37106
## 13 g1_41/g1_46.17 46.91099
## 14 g1_41/g1_46.88 52.91840
```

```
## 15 g1_42/g1_46.42 49.43964
```

```
#The selected individuals are much better than the initial parents
#1) Calculate breeding values
BVsParents <- c(Markers_P%*%markereffectsAll)
BVsOpt <- c(Markers_Offspring[out$OptimalSelection$Genotypes,]%*%markereffectsAll)

#Original parental population
mean(BVsParents) #average
```

```
## [1] -29.94445
```

```
max(BVsParents) #best
```

```
## [1] 18.37439
```

```
#Selected Offspring
mean(BVsOpt) #much larger than parental average
```

```
## [1] 52.25871
```

```
max(BVsOpt) #much better than the best parent
```

```
## [1] 61.74161
```

Selecting the best offspring this way has a few potential **advantages over truncation selection**. Firstly, the individuals selected by **truncation selection** will all **perform very well**, but **most will be redundant** as they are likely to be **extremely similar to one another**. Secondly, our **estimated marker effects** and breeding values can have **some error**. Selecting **only the best individuals** according to them is extremely **risky**, as it is possible that what we are selecting are not the the individuals with the actual best true breeding values. Using **optimization** to select a more **diverse** set of individuals mitigates this risk, as a **better screening** of the available genotypes is performed.

Post-Optimization, reference population

Post-optimization is very flexible and can be used in many different scenarios. For instance, it is possible that full genomic mating (optimizing the pairing between parents) is not being applied in a breeding program, but it is desired to optimize the composition of the parental pool for the next generation. This can be done with two changes:

1) Let the candidates for selection be the existing parental lines plus the candidate offspring for recycling. Also, set the number of selected individuals to the desired size of the parental pool (usually kept constant and equal to the size of the old parental pool). The solution of the optimization will contain the new parental pool, in which some old parents are replaced by candidate offspring.

2) Compute diversity loss with respect to the original pool of parental lines, which measures the overall reduction in genetic variability after recycling. This can be done by selecting all parents as the reference population.

```
set.seed(12)
out <- PostOpt(Markers_Cand = rbind(Markers_P, #markers of original parental pool
                                    Markers_Offspring), #markers of offspring
               Reference_population = Markers_P, #we want to compute PropSD with respect
               #to the parental pool. We can include the marker matrix or a vector of names
               parametrization = "Genotypic",
               phi = phi,
               size = nrow(Markers_P), #keep size of parental pool constant
               markereffects = markereffects,
               markereffects_d = NULL, #self-pollinated --> no dominance
```

```
                coefficients = coefficients,
                PropSD = PropSD_limit,
                control = control,
                Username=username, #you can get one by contacting us
                Password=password, #you can get one by contacting us
                Username_TrainSel=username_TrainSel, #you can get one by contacting us
                Password_TrainSel=password_TrainSel) #you can get one by contacting us)
```

```
PropSD_limit #desired limit to diversity loss
```

```
## [1] 0.05
```

```
out$PropSD #below the desired cutoff
```

```
## [1] 0.04964508
```

```
#Selected individuals
head(out$OptimalSelection) #the Fitness column is simply the breeding value
```

```
##   Genotypes    Fitness
## 1      g1_2 -27.841081
## 2      g1_3  -5.255084
## 3      g1_4 -11.571864
## 4      g1_6 -26.467708
## 5      g1_7  -7.959797
## 6      g1_8 -46.408799
```

```
#number of old parents kept:
sum(out$OptimalSelection$Genotypes %in% rownames(Markers_P))
```

```
## [1] 13
```

```
#Number of new genotypes included into the parental pool:
sum(out$OptimalSelection$Genotypes %in% rownames(Markers_Offspring))
```

```
## [1] 37
```

```
#Less aggressive recycling can be obtained by reducing PropSD

#The selected individuals are much better than the initial parents
#1) Calculate breeding values
BVsParents <- c(Markers_P%*%markereffectsAll)
BVsOpt <- c(rbind(Markers_Offspring,Markers_P)[out$OptimalSelection$Genotypes,]%*%markereffectsAll)

#Original parental population
mean(BVsParents) #average
```

```
## [1] -29.94445
```

```
max(BVsParents) #best
```

```
## [1] 18.37439
```

```
#New parental population
mean(BVsOpt) #much larger than parental average
```

```
## [1] 25.04996
```

```
max(BVsOpt) #much better than the best parent
```

```
## [1] 61.74161
```

```
#Selection intensity for the breeding cycle (in units of additive SD):
(mean(BVsOpt) - mean(BVsParents))/sd(BVsParents)
```

```
## [1] 1.987799
```

```
#associated loss of additive standard deviation:
out$PropSD
```

```
## [1] 0.04964508
```

In summary, using the reference population it is possible to perform very flexible optimizations. Here, we found the best composition of the new parental pool that included a mixture of old parental lines and new offspring, obtaining the best possible balance between genetic gain and preservation of variance according to the desired value of PropSD. We only optimized the composition of the new parental pool, but not the pairing between them. For that, we would need to use the `GenomicMatingMT()` function as showcased above.

Post-Optimization, hybrid crop

Here, we will use the same parameters as before, but for selecting the best 15 offspring while maintaining diversity. We will **evaluate** them according to the expected **performance of the hybrids they generate when crossed with testers**. As a result, we need to take **dominance effects** into account.

```
phi = 2 #diploid crop
size = 15 #we want to select 15 individuals
#limit diversity loss:
PropSD_limit <- 0.05
#we want to maximize the average genotypic values of the hybrid generated by the
#selected individuals. Thus, TestersApproach = "average". It would also be possible
#to set TestersApproach = "max" to select individuals according to the genotypic
#value of the best hybrid they generate.
TestersApproach = "average"

#Demo TrainSel settings, for speed (see the note at the start of the vignette).
control = TrainSel::SetControlDefault(size="demo",
                                      verbose = F)

# #Recommended parameters (slower but converge to a much better solution):
# control = TrainSel::SetControlDefault(size="large",
#                                       complexity = "high_complexity",
#                                       verbose = F)
```

```
set.seed(12)
out <- PostOpt(Markers_Cand = Markers_Offspring,
               Markers_T = Markers_potential_testers,
               parametrization = "Genotypic",
               TestersApproach = TestersApproach,
               phi = phi,
               size = size,
               markereffects = markereffects,
               markereffects_d = markereffects_d, #hybrid --> we need dominance
               coefficients = coefficients,
               PropSD = PropSD_limit,
               control = control,
               Username=username, #you can get one by contacting us
               Password=password, #you can get one by contacting us
```

```
                Username_TrainSel=username_TrainSel, #you can get one by contacting us
                Password_TrainSel=password_TrainSel)
```

```
PropSD_limit #desired limit to diversity loss
```

```
## [1] 0.05
```

```
out$PropSD #below the desired cutoff
```

```
## [1] 0.04264921
```

```
#Selected individuals
#The negative fitness of the selected lines can be explained because, in this
#example dataset, the specific combining ability with the testers was very poor.
out$OptimalSelection
```

```
##          Genotypes Recommended_Tester    Fitness
## 1     g1_4/g1_43.2              g2_35  1.2036184
## 2   g1_14/g1_15.37              g2_49 -1.0884335
## 3   g1_14/g1_34.54              g2_36  2.3621168
## 4   g1_14/g1_42.43              g2_40  2.1385734
## 5   g1_14/g1_42.41               g2_8 17.0946115
## 6   g1_14/g1_42.57               g2_7 17.4829581
## 7   g1_14/g1_42.25              g2_34  0.2849694
## 8    g1_14/g1_46.5              g2_21  1.1034215
## 9   g1_14/g1_46.28              g2_49 -0.8678944
## 10  g1_15/g1_43.36              g2_13  1.8535998
## 11 g1_26/g1_46.128               g2_2 -0.7544374
## 12  g1_34/g1_42.41              g2_28 -1.5009414
## 13  g1_39/g1_42.65              g2_26  0.9571335
## 14  g1_42/g1_46.42              g2_47 -1.5707835
## 15  g1_42/g1_46.67              g2_14  3.8890750
```

```
#The selected individuals are much better than the initial parents
#1) Calculate breeding values (General Combining Ability)
BVsParents <- c(Markers_P%*%markereffectsAll)
BVsOpt <- c(Markers_Offspring[out$OptimalSelection$Genotypes,]%*%markereffectsAll)

#Original parental population (General Combining Ability)
mean(BVsParents) #average
```

```
## [1] -29.94445
```

```
max(BVsParents) #best
```

```
## [1] 18.37439
```

```
#Selected Offspring (General Combining Ability)
mean(BVsOpt) #much larger than parental average
```

```
## [1] 47.09218
```

```
max(BVsOpt) #much better than the best parent
```

```
## [1] 59.96783
```

The output of the optimization is similar to what we had in the self-pollinated crop with a few differences. For hybrids, we have an additional column with the **recommended tester**, i.e., the tester that is expected to generate the best hybrid when crossed with the selected genotype. Furthermore, the **"Fitness"** column

now contains the average **genotypic value for the hybrids** that can be created by crossing each genotype with all testers (when `TestersApproach = "average"` is used). Alternatively, it can contain the genotypic value of the hybrid obtained with the recommended tester specifically if `TestersApproach = "max"` is used.

Advantages and Limitations

In this section, we will summarize some of the key characteristics that make MateR appealing. To find the theoretical background that supports our claims, please see the MateR publication. The main **advantages** of the MateR package are the following:

1. It is highly **versatile and easy to use**, accommodating for numerous breeding schemes.
2. It supports **multi-trait** analysis.
3. MateR accepts both **proportion of additive standard deviation lost** and **inbreeding rate** as **parameters controlling** the maximum acceptable **diversity loss**. The latter is the most typical one, but its computation from a genomic relationship matrix incurs in some error and it is difficult to directly link it to the rate of reduction in genetic gain. Conversely, The former has been newly developed to solve these issues and we recommend its use. In any case, both parameters rely on metrics frequently used by breeders, which facilitates the choice of its desired value by the users.
4. Highly **informative and readable output**.
5. The **usefulness of each cross is calculated integrating a lot of information**, allowing to account for **selfing**, **number of offspring** generated per cross, **additive** and **dominance** effects, directional dominance (i.e., **heterosis and inbreeding depression**), **interaction between additive and dominance** effects within a locus, **interaction between** marker effects of **correlated traits** and **linkage disequilibrium**.
6. **Diploids, allopolyploids and autotetraploids** are supported.
7. Genomic mating requires finding the best crosses among a very large set of possibilities, making optimization often very time-consuming. MateR has a **computationally efficient implementation**. Some alternatives, like convex optimization, can be substantially faster. However, convex optimization only works for a convex objective function. This could be possible if the way in which usefulness is computed is substantially simplified, but that could have negative impacts on the quality of the results.
8. **Additional utility functions** are provided to facilitate the calculation of the inputs needed for genomic mating. For instance, `meff_from_GVs()` allows to easily compute the marker effects.
9. MateR supports both the **"Genotypic"** and **"Breeding" parametrizations** of marker effects and marker scores.

However, it also has some **limitations**:

1. **Epistasis is disregarded**.
2. **Allopolyploids are treated as diploids**. Therefore, the **dominance between different subgenomes** is modeled as epistasis, which **is not considered** in MateR.
3. Calculating within-family variance can be very **computationally intensive if haplotypes are considered**. However, MateR provides much faster alternatives that can provide comparable accuracy, mitigating this issue. If computational time is a bottleneck, we generally recommend using the simplification `LD="Approx"`.
4. Autotetraploids are supported, but **other autopolyploids are not**.
5. MateR **requires license keys**, but they will be provided for **free to public bodies**, such as Universities and non-profit organizations. You can get license keys by contacting javfer98@gmail.com or j.isidro@upm.es.

References

Fernández-González, J., Metwally, S. M., & Isidro y Sánchez, J. (2026). MateR: a novel genomic mating framework. Genetics, 232(4), iyag013. https://doi.org/10.1093/genetics/iyag013

Metwally, S. M., Fernández-González, J., & Isidro y Sánchez, J. (2025). Expanding the gain-variance Pareto via optimal recycling and genomic mating. bioRxiv, 2025-09.

Jeffrey B Endelman, Genomic prediction of heterosis, inbreeding control, and mate allocation in outbred diploid and tetraploid populations, Genetics, Volume 229, Issue 2, February 2025, iyae193